



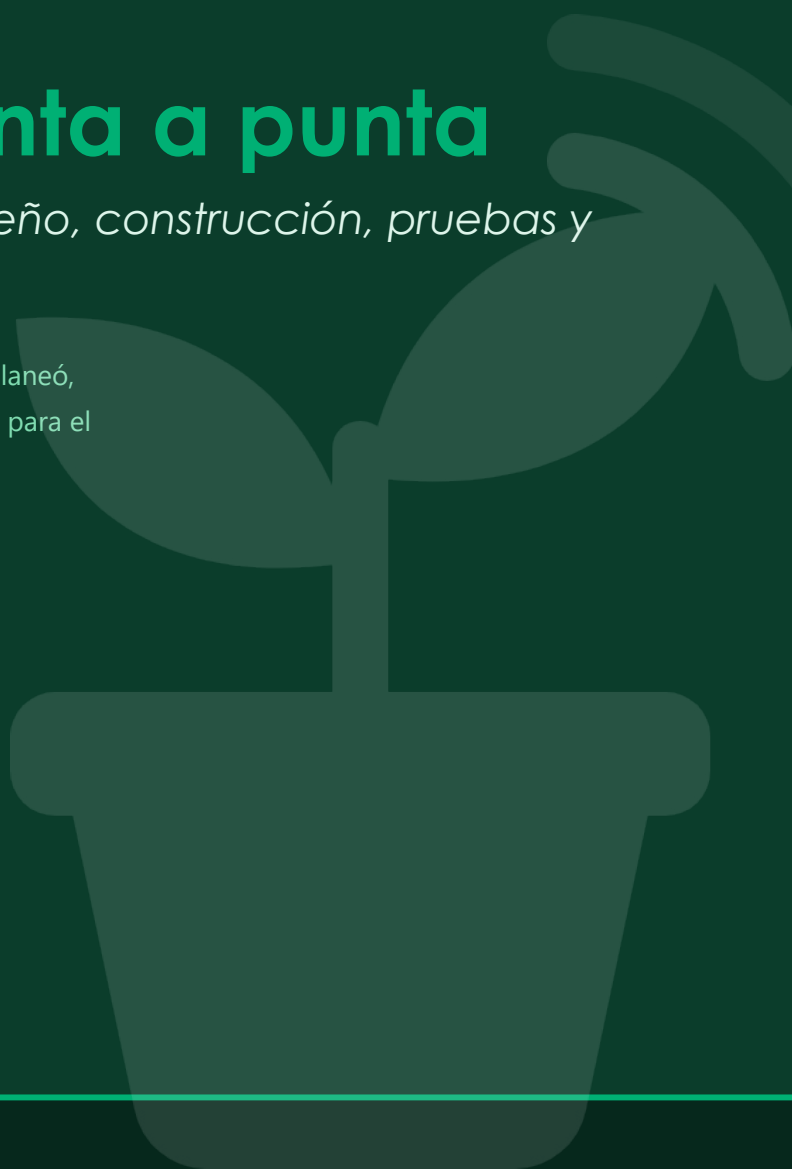
SmartPot

CICLO DE VIDA DEL SOFTWARE

SmartPot de **punta a punta**

Formulación, gestión, análisis, diseño, construcción, pruebas y mejora continua

Una lectura crítica de cada etapa del proyecto: qué se planeó, qué evidencia dejó, qué no cuadraba y qué se aprendió para el siguiente ciclo.



DOCUMENTO

Ciclo de vida del software

VERSIÓN

1.0 · septiembre 2026

EQUIPO

SmartPotTech

PROYECTO

smartpot.app



DOCUMENTO

Ficha del documento

CAMPO	VALOR
Proyecto	SmartPot · smartpot.app
Organización	SmartPotTech
Documento	Ciclo de vida del software: formulación y evaluación, gestión, análisis, diseño, construcción, pruebas y mejora continua
Versión	1.0 · septiembre 2026
Periodo	Septiembre de 2024 a septiembre de 2026
Fuentes	Formulación del proyecto (idea, problema, árboles, 5 porqués, PESTEL, involucrados, marco lógico), estudio de mercado, acta de constitución, informe de gestión del cronograma y los recursos, informe de tareas críticas, propuesta de investigación, requisitos, casos de uso, diagramas UML del diseño y base de conocimiento; todo contrastado con el historial de los repositorios y la plataforma en producción
Documentos hermanos	Documentación técnica : la plataforma tal como funciona hoy. Recorrido del proyecto : requisito por requisito y diagrama por diagrama, qué se planeó y qué quedó construido
Cómo leerlo	Cada parte corresponde a una disciplina del ciclo de vida y tiene tres capas: lo que se hizo, con su diagrama; la lectura crítica , con lo que estuvo bien y lo que no cuadraba; y la evidencia hoy , con lo que queda en el código, las pruebas o la operación

SECCIÓN 01

El proyecto en el tiempo

En palabras simples

SmartPot pasó por todas las etapas de un proyecto de software, aunque no en el orden de un libro. Primero se diseñó y se construyó una primera versión; después se formuló como proyecto (problema, objetivos, mercado y finanzas); luego se planeó con las herramientas de la gerencia de proyectos; y al final la plataforma se reestructuró en ciclos cortos de mejora continua hasta quedar en producción. Este documento recorre esas etapas como disciplinas y, en cada una, separa lo que se dijo de lo que se puede comprobar.

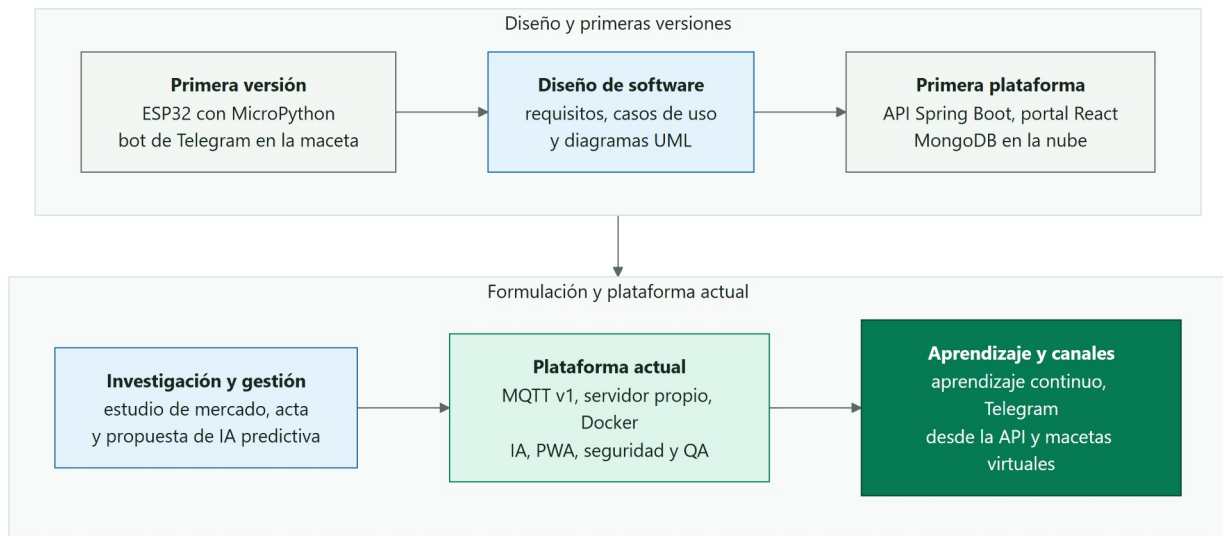


Figura 1 · Etapas del proyecto

SEMESTRE	DISCIPLINAS DOMINANTES	QUÉ PRODUJO	COMMITTS HUMANOS
2024-2	Análisis y diseño	Primera matriz de requisitos (agosto de 2024), casos de uso, diagramas UML, primera API, primer portal, firmware del ESP32, simulador y primeros análisis de datos	379
2025-1	Construcción	Ajustes y mantenimiento de la primera plataforma	141
2025-2	Formulación y evaluación	Idea, problema, árboles, marco lógico, estudio de mercado (noviembre de 2025) y crecimiento de la PWA	553
2026-1	Gestión de proyectos	Acta de constitución, EDT, cronograma con ruta crítica, recursos y nivelación; propuesta de investigación	145
2026-2	Construcción, pruebas y mejora continua	Reestructuración completa, tres ciclos de mejora y la plataforma en producción	714

Los repositorios suman 2217 commits, de los cuales 1932 son de personas y el resto de bots de automatización, sobre todo actualizaciones de dependencias. La tabla ya deja ver la primera lección del proyecto: los semestres de planeación formal (2026-1) fueron los de menos construcción, y el mayor avance llegó cuando el trabajo se organizó en ciclos cortos con pruebas automáticas.

DISCIPLINA	PARTE	DIAGRAMAS
Formulación y evaluación de proyectos TI	I	Árbol de problemas, causa y efecto, árbol de objetivos, poder e interés, fases de la investigación
Gestión de proyectos TI	II	EDT, red de precedencias, ruta crítica, ciclo Scrum, riesgos
Análisis	III	Casos de uso



DISCIPLINA	PARTE	DIAGRAMAS
Diseño	IV	Contexto, contenedores, componentes, clases, actividad, secuencia, estados, datos, mapa de la PWA, redes
Construcción	V	Despliegue, asistente de IA, aprendizaje, Telegram, maceta virtual
Pruebas	VI	Pirámide de pruebas, workflow de QA
Mejora continua	VII	Ciclo PDCA, ciclos de mejora

CONTENIDO

Contenido

1. El proyecto en el tiempo

2. Identificación del problema

- 2.1 La idea
- 2.2 Árbol de problemas
- 2.3 Causa y efecto
- 2.4 Los 5 porqués

3. Objetivos y alternativa elegida

- 3.1 Árbol de objetivos
- 3.2 Objetivo SMART
- 3.3 Cómo evolucionaron los objetivos

4. Entorno e involucrados

- 4.1 PESTEL
- 4.2 Poder e interés

5. Marco lógico

6. Estudio de mercado

- 6.1 Demanda y oferta
- 6.2 Coherencia de las cifras
- 6.3 Qué quedó del estudio

7. Estudio técnico

8. Estudio financiero y evaluación

- 8.1 Presupuestos
- 8.2 Modelo de evaluación

9. Propuesta de investigación

10. Acta de constitución: lo que fijó y lo que pasó

11. Alcance

- 11.1 Estructura de desglose del trabajo
- 11.2 Priorización y MVP



12. Cronograma

12.1 Red de precedencias

12.2 Ruta crítica

13. Recursos, asignación y nivelación

14. Riesgos

15. Metodología, comunicaciones y control de cambios

16. De la necesidad al requisito

17. Casos de uso

18. Trazabilidad

19. Reglas de negocio y base de conocimiento

20. Arquitectura

20.1 Contexto

20.2 Contenedores

20.3 Decisiones de arquitectura

21. Del diseño original al actual

21.1 Clases

21.2 Componentes

21.3 Comportamiento: enviar un comando

21.4 Comportamiento: datos históricos y una lectura

21.5 Actividad: control automático

21.6 Datos

21.7 Objetos y artefactos

22. Interfaz

23. Seguridad por diseño

24. Organización del trabajo

25. Infraestructura y despliegue

26. Asistente de IA

27. Integraciones

27.1 Telegram

27.2 Macetas virtuales

28. Aprendizaje continuo

29. Estrategia

30. Criterios de aprobación del acta

31. Calidad de los modelos

32. El ciclo de mejora

33. Ciclos de mejora



34. Mejora que ocurre sola

35. Indicadores

36. Backlog del próximo ciclo

37. Lecciones aprendidas



PARTE I

Formulación y evaluación de proyectos TI

SECCIÓN 02

Identificación del problema

2.1 La idea

SmartPot fue la idea elegida entre las propuestas de proyecto de un curso de tecnologías de la información. Nació de una observación simple: el riego y la nutrición de un cultivo hidropónico dependen de variables que cambian todo el tiempo (pH, sólidos disueltos, luz, temperatura y humedad), y la mayoría de pequeños productores y aficionados las controla a mano. La idea citaba tres datos de la FAO: la agricultura usa cerca del 70 % del agua dulce, el riego tradicional pierde hasta el 40 % por ineficiencia y la hidroponía puede reducir el consumo hasta en un 90 %.

La idea se formuló como un proyecto de datos y no como un aparato:

ALCANCE DE LA IDEA	CÓMO QUEDÓ CONSTRUIDO
Captura en tiempo real desde sensores	Telemetría MQTT v1 con credencial por maceta
Integración y estandarización de las mediciones	Contrato JSON único y validadores <code>\$jsonSchema</code> en MongoDB
Automatización del riego según lo sensado	Agente de automatización con modo automático y enfriamiento por actuador
Visualizaciones, tabulados y alertas	PWA con panel general, historial, comparador y alertas por Telegram
Diccionario de variables y metadatos	Contrato MQTT, esquema de datos y base de conocimiento documentados
Documentación reproducible	Tres documentos, diagramas en Mermaid y una demo de un solo comando

LECTURA CRÍTICA

Plantear SmartPot como un *pipeline* de datos (capturar, limpiar, integrar, decidir, mostrar) fue la mejor decisión de la formulación. Separó el valor del hardware: la maceta solo mide y obedece, y todo lo que aprende la plataforma sirve igual para una maceta física, una simulada en Wokwi o una virtual.

2.2 Árbol de problemas

El problema central se definió como la **gestión ineficiente de cultivos hidropónicos por falta de control preciso y continuo de sus parámetros**.

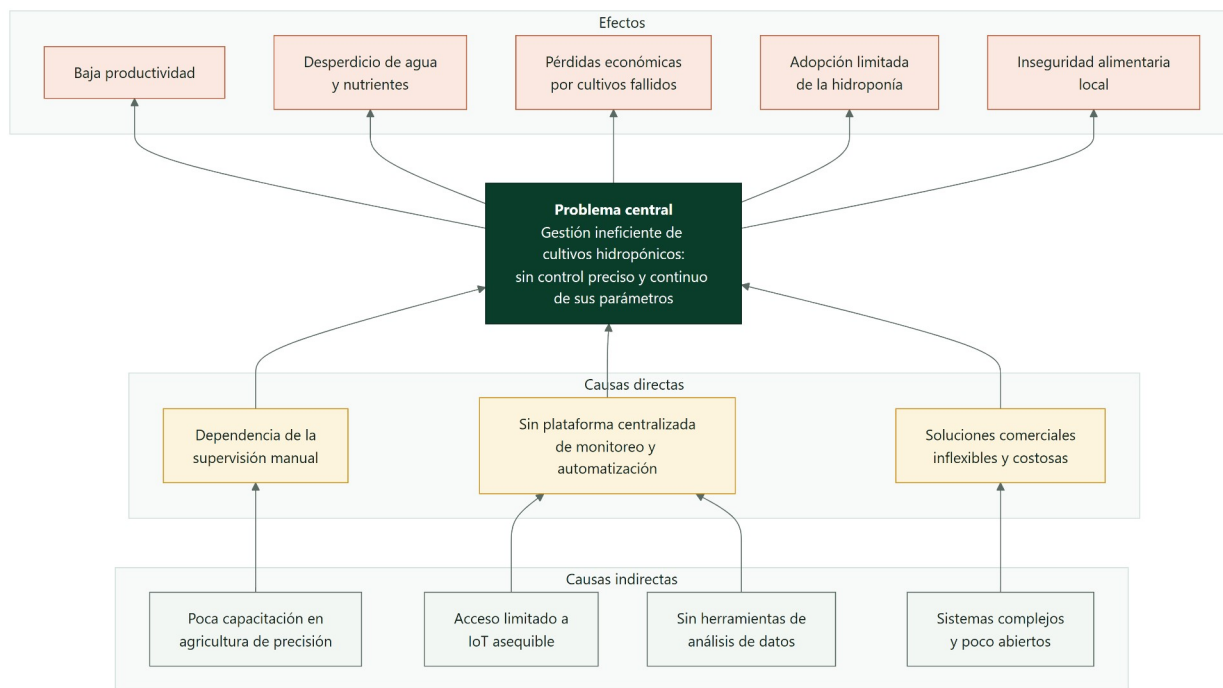


Figura 2 · Árbol de problemas

2.3 Causa y efecto

El diagrama de Ishikawa ordenó las causas con las categorías clásicas de métodos, máquinas, mediciones, mano de obra y medio ambiente.

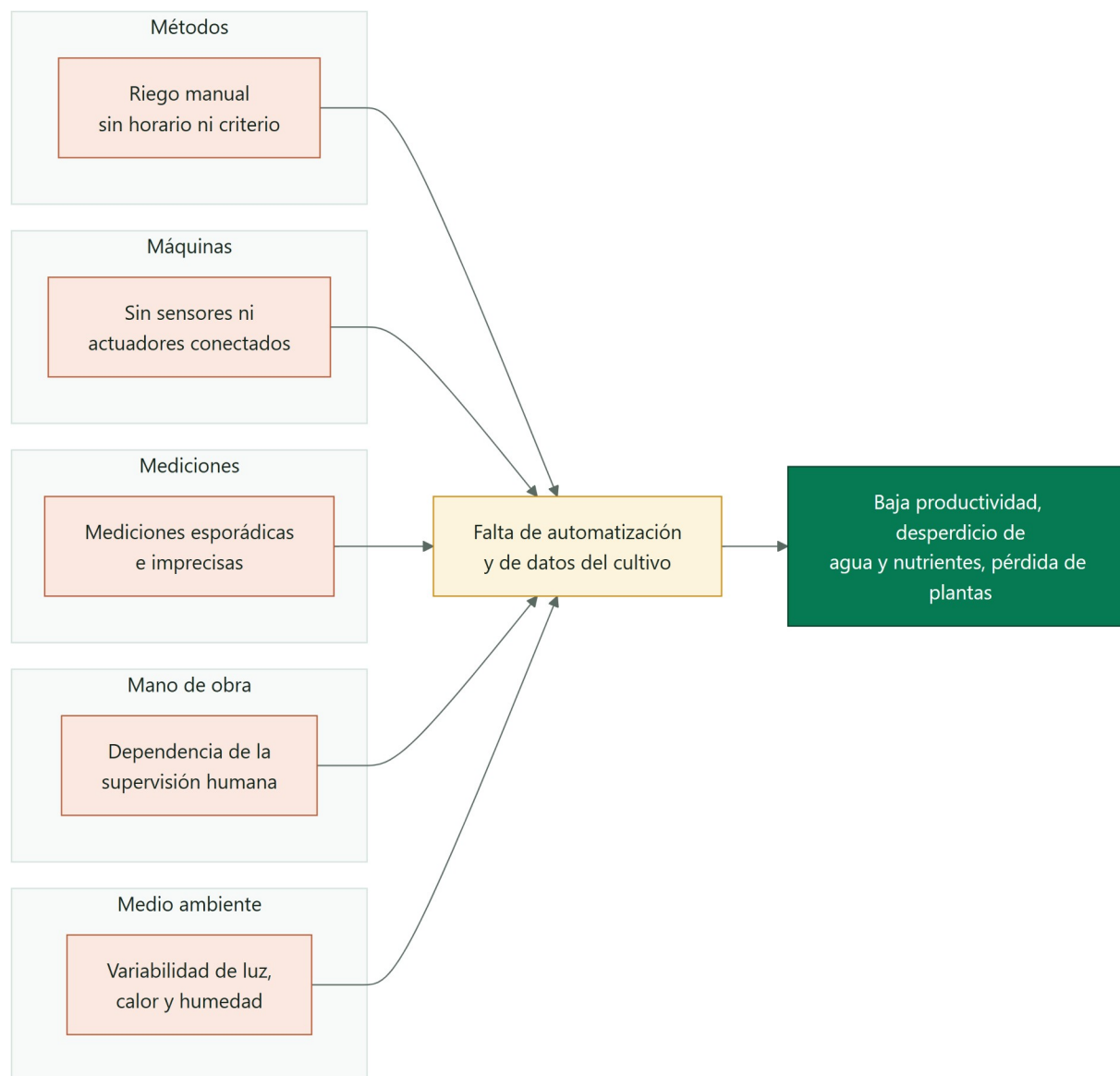


Figura 3 · Causa y efecto del problema

2.4 Los 5 porqués

PREGUNTA	RESPUESTA DE LA FORMULACIÓN
¿Por qué la gestión es ineficiente?	Porque depende de la supervisión manual y hay poco control automatizado
¿Por qué la supervisión manual desperdicia recursos?	Porque una persona no puede seguir todas las variables todo el tiempo
¿Por qué no se automatiza?	Porque las soluciones son costosas, poco accesibles o complejas para pequeños productores



PREGUNTA	RESPUESTA DE LA FORMULACIÓN
¿Por qué el costo y la rigidez son un obstáculo?	Porque exigen una inversión alta y no se adaptan a cada cultivo
¿Por qué crear SmartPot?	Para ofrecer una herramienta asequible, flexible y educativa que use la simulación para bajar costos

LECTURA CRÍTICA

Las tres técnicas llegan a la misma raíz, falta de automatización y de datos del cultivo, y eso da confianza en el diagnóstico. Tienen tres debilidades que conviene corregir en una próxima formulación:

- El quinto porqué no es una causa sino la justificación de la solución; la cadena causal termina en el cuarto.
- El Ishikawa omite la categoría *materiales* (solución nutritiva, sustrato, calidad del agua), que en hidroponía explica buena parte de las pérdidas.
- Efectos como la inseguridad alimentaria local son reales pero el proyecto no puede medirlos; en el marco lógico quedan como fin, no como indicador.

SECCIÓN 03

Objetivos y alternativa elegida

3.1 Árbol de objetivos

Al invertir el árbol de problemas, cada causa se convierte en un medio. El diagrama agrega, en cursiva, cómo construyó SmartPot cada medio.

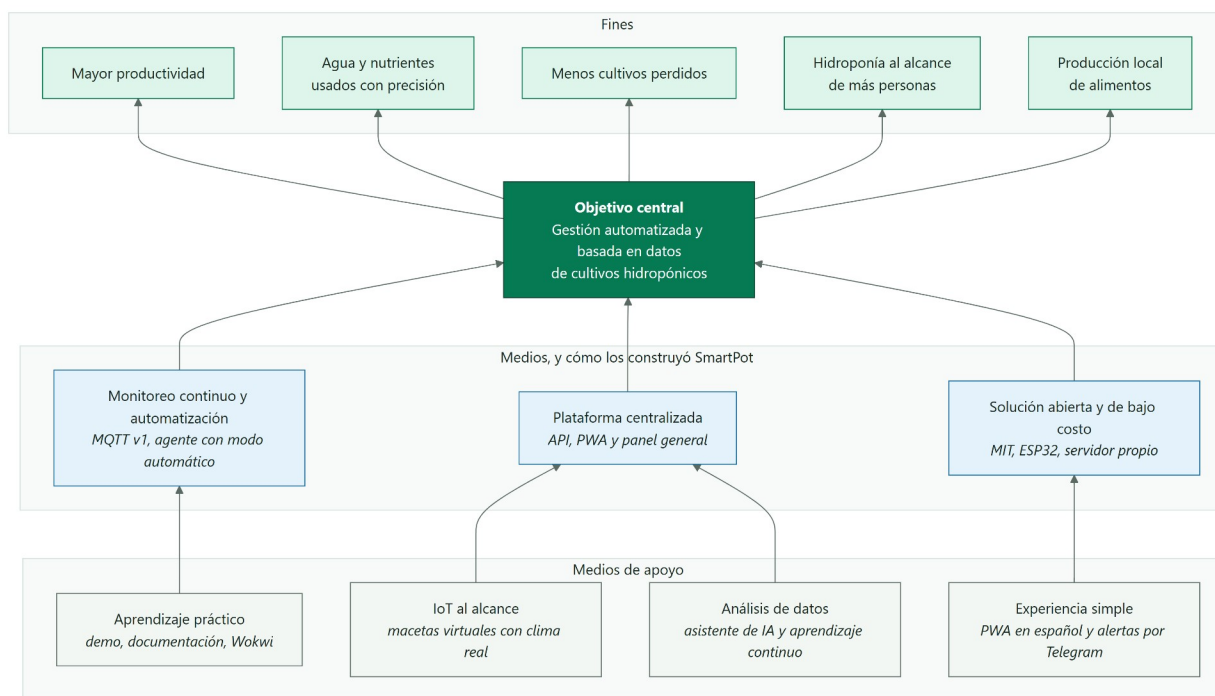


Figura 4 · Árbol de objetivos

3.2 Objetivo SMART

CRITERIO	FORMULACIÓN	LECTURA CRÍTICA
Específico	Monitorear y automatizar un jardín hidropónico simulado: humedad, temperatura, luz, pH y nutrientes	Bien acotado; la plataforma lo cumple y agrega humedad del sustrato y atmósfera
Medible	Simulación completa en Wokwi, API, interfaz, base de datos, reportes y riego e iluminación automáticos	Mide productos , no resultados: no dice cuánta agua se ahorra ni cuánto tiempo pasa el cultivo en rango
Alcanzable	Prototipo dentro del curso con un proceso iterativo	Cierto para el software; la parte física quedó fuera por diseño
Relevante	Valor académico y social	Se sostiene: código abierto, demo para instituciones y documentación completa
Temporal	Análisis y diseño de agosto a septiembre; desarrollo y pruebas de octubre a diciembre	La primera plataforma existía antes de esta formulación y siguió creciendo dos años más; el plazo describía un semestre, no el producto

3.3 Cómo evolucionaron los objetivos

DOCUMENTO	OBJETIVO GENERAL	QUÉ CAMBIÓ
Formulación	Automatizar la gestión de cultivos hidropónicos con monitoreo en tiempo real y análisis de datos	Cinco objetivos específicos por etapa: analizar, diseñar, desarrollar, probar y mantener
Proyecto integrador	El mismo	Siete objetivos por beneficio: monitoreo remoto, agua y nutrientes, errores humanos, pérdida de plantas, dependencia manual, espacios reducidos e historial
Acta de constitución	Sistema completo de monitoreo y automatización para lechuga y tomate	Objetivos por dimensión (alcance, tiempo, costo, calidad y relevancia) con indicadores de éxito
Propuesta de investigación	Integrar IoT y modelos de analítica para el control inteligente de la lechuga	Seis objetivos que incluyen hardware físico, parámetros agronómicos y validación experimental

LECTURA CRÍTICA

El objetivo general se mantuvo estable durante dos años, lo que muestra una visión clara. Los específicos, en cambio, se reescribieron en cada curso sin trazar cuáles reemplazaban a cuáles. La plataforma cumple los objetivos de software de los cuatro documentos; quedan abiertos los que exigen hardware y un ciclo de cultivo real (sección 9).

SECCIÓN 04

Entorno e involucrados

4.1 PESTEL

FACTOR	FORMULACIÓN	HOY
Político	Políticas de agricultura urbana y apoyo institucional	Sin cambios: el proyecto sigue dentro del marco académico
Económico	Herramientas educativas gratuitas y alojamiento económico	Se reemplazaron las capas gratuitas por un servidor propio pequeño para no depender de cuotas
Social	Seguridad alimentaria, aprendizaje práctico y comunidades <i>maker</i>	Demo de un solo comando y macetas virtuales para aprender sin hardware
Tecnológico	Wokwi, Spring Boot, React y ESP32	Se sumaron MQTT con TLS, Docker, FastAPI con scikit-learn y la PWA instalable
Ecológico	Eficiencia hídrica y menos pesticidas	Riego preventivo y bloqueo de nutrientes; el ahorro real solo se podrá medir con un prototipo físico



FACTOR	FORMULACIÓN	HOY
Legal	Marco académico y software libre	Licencia MIT, datos personales mínimos, borrado de la cuenta y cultivos seudonimizados para el aprendizaje

4.2 Poder e interés

El análisis identificó actores primarios (docente y equipo de desarrollo), secundarios (institución, coordinación, proveedores de nube, horticultores, microproductores y comunidad *maker*) y terciarios (usuarios potenciales, proveedores alternativos y competencia). La matriz de interesados calificó a nueve de ellos de 1 a 10 en poder e interés.

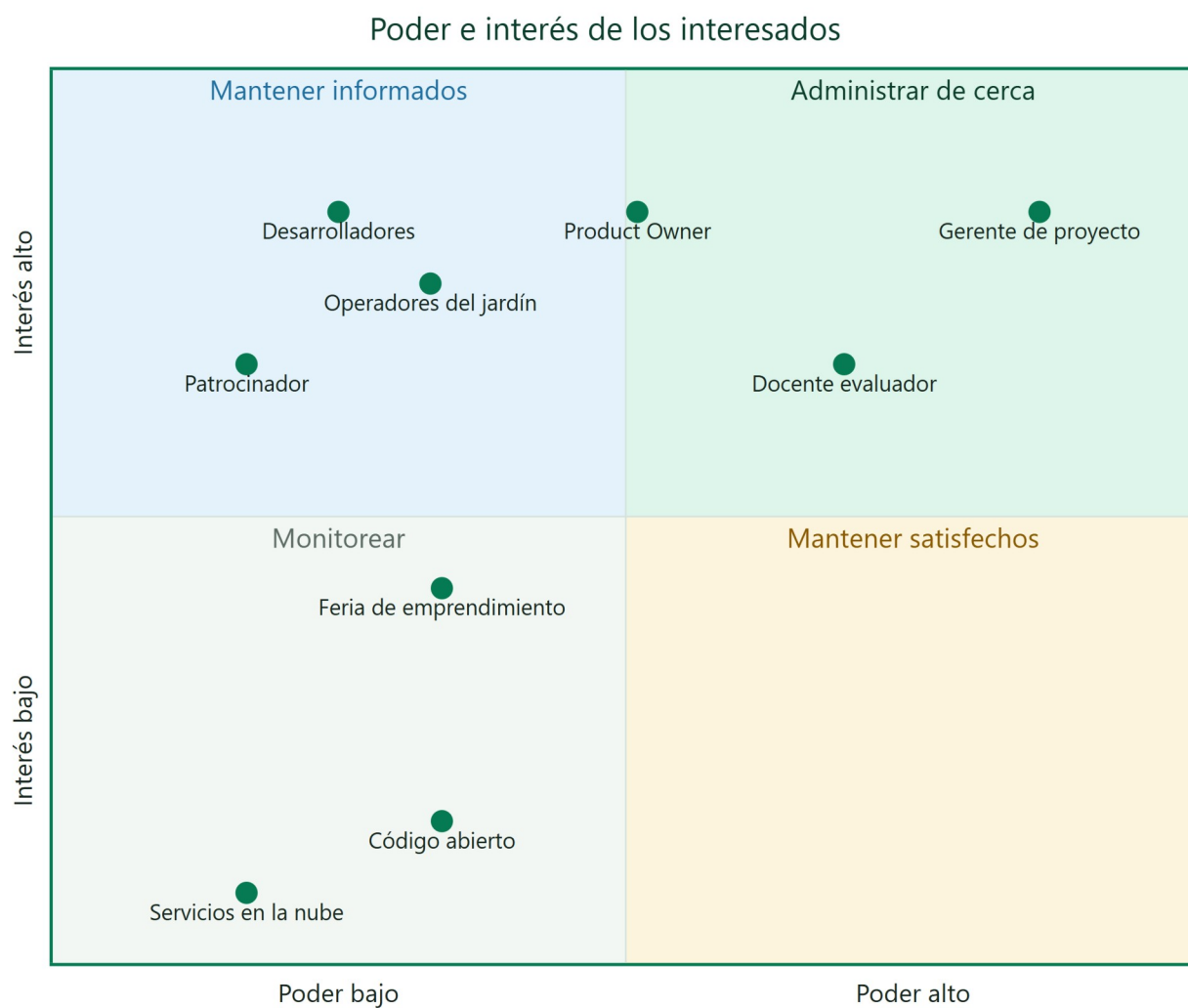


Figura 5 · Poder e interés de los interesados



INTERESADO	PODER	INTERÉS	IMPACTO (PODER × INTERÉS)	MANEJO SEGÚN SU POSICIÓN	MANEJO EN LA MATRIZ ORIGINAL
Gerente de proyecto	10	10	100	Administrar de cerca	Administrar de cerca
Docente evaluador	8	8	64	Administrar de cerca	Administrar de cerca
Product Owner	6	10	60	Administrar de cerca	Administrar de cerca
Desarrolladores	3	10	30	Mantener informados	Mantener satisfechos
Operadores del jardín	3	9	27	Mantener informados	Mantener satisfechos
Institución patrocinadora	2	8	16	Mantener informados	Mantener satisfechos
Feria de emprendimiento	4	5	20	Monitorear (en el límite)	Monitorear
Comunidad de código abierto	4	2	8	Monitorear	Monitorear
Servicios en la nube	2	1	2	Monitorear	Monitorear

LECTURA CRÍTICA

Tres de los nueve interesados tenían un manejo que no correspondía a su posición: con poder bajo e interés alto, la estrategia es *mantener informados*, no *mantener satisfechos*. Además, la columna de conclusiones quedó desplazada una fila y trae frases de otro contexto (ventas, una aplicación móvil, un equipo interno de TI), y la exposición oral ubicó a los proveedores de nube con poder alto mientras la matriz les daba 2. La tabla anterior es la versión corregida y es la que usa el [recorrido del proyecto](#).

SECCIÓN 05

Marco lógico

NIVEL	RESUMEN NARRATIVO	INDICADOR PLANTEADO	EVIDENCIA HOY
Fin	Contribuir a la sostenibilidad y eficiencia de la agricultura urbana mediante la tecnología	Menor consumo de agua; más uso de tecnologías de monitoreo	No medible todavía: requiere un prototipo físico y un ciclo de cultivo comparado
Propósito	Optimizar la gestión de cultivos hidropónicos con monitoreo y control automatizados	95 % de precisión en la gestión de variables; menos tiempo de supervisión manual	El agente actúa solo con el modo automático; «precisión» nunca se definió, así que no hay con qué comparar



NIVEL	RESUMEN NARRATIVO	INDICADOR PLANTEADO	EVIDENCIA HOY
Resultado 1	Prototipo completo en simulación	Validación en Wokwi	Cumplido: firmware en Wokwi y macetas virtuales que publican por MQTT
Resultado 2	Plataforma web con backend y frontend	API con 80 % de cobertura de pruebas	Plataforma en producción con 260 pruebas; la cobertura no se mide formalmente
Resultado 3	Monitoreo y control en tiempo real	Sistema funcional	Cumplido: telemetría, comandos con confirmación y estado en línea
Actividades	Análisis y diseño; desarrollo; pruebas y validación; documentación y cierre	Cronograma y repositorios	Cumplidas en ciclos (parte VII)

LECTURA CRÍTICA

El marco lógico tenía la estructura correcta pero indicadores sin línea base, sin meta fechada y, sobre todo, sin supuestos: la columna que dice qué debe ser cierto fuera del proyecto para que funcione. Para el próximo ciclo se propone medir el propósito con un indicador que la plataforma ya puede calcular:

- **Tiempo en rango:** porcentaje de lecturas de cada cultivo dentro del rango ideal de su especie, por semana.
- **Intervenciones manuales:** comandos enviados por la persona frente a los ejecutados por el agente.
- **Supuestos explícitos:** la maceta tiene conexión estable, los sensores están calibrados y la persona activa el modo automático.

SECCIÓN 06

Estudio de mercado

6.1 Demanda y oferta

SEGMENTO	DEMANDA INICIAL ESTIMADA
Horticultores urbanos y domésticos (Medellín, Bogotá, Cali)	500 a 2000 personas
Instituciones educativas y semilleros	50 a 150 instituciones
Makers y comunidad IoT	200 a 500 personas
Microproductores agrotecnológicos	100 a 300 personas

La oferta se describió como un oligopolio parcial con tres tipos de oferentes: sistemas comerciales de hidroponía inteligente (500 a 5000 USD, robustos pero caros), plataformas IoT genéricas (baratas pero sin



el dominio hidropónico) y desarrollos caseros (gratuitos pero sin soporte). SmartPot se ubicó en el espacio vacío entre ellos: accesible, educativo y listo para usar.

6.2 Coherencia de las cifras

DATO	PRESENTACIÓN DEL ESTUDIO	DOCUMENTO DEL ESTUDIO	OBSERVACIÓN
Plan Profesional	9 a 12 USD al mes	20 USD al mes	Dos precios para el mismo plan
Plan Empresarial	50 a 100 USD al mes	100 USD al mes	Coherente en el tope
Costo del MVP	~20 USD al año	81 000 a 120 000 COP al año	El mismo documento lista un servidor de 5 a 8 USD al mes, que ya suma 60 a 96 USD al año
Crecimiento	30 % a 40 % anual	De 800–2900 a 2500–5000 personas	Pasar de un rango al otro implica crecer entre 72 % y 212 %
Demanda	Consumo nacional aparente	Producción + importaciones – exportaciones	Fórmula de bienes físicos; no se calculó para un servicio digital
Aceptación	75 % a 85 %	Probabilidad «alta»	Sin método ni fuente primaria
Valor percibido	—	Ahorro de agua del 90 %	Es un beneficio de la hidroponía, no de SmartPot

LECTURA CRÍTICA

El estudio acertó en lo cualitativo: el nicho entre lo caro y lo casero existe, el canal académico es el natural y el costo de operación es bajo. Falló en lo cuantitativo: los rangos de demanda no salen de un cálculo, el precio cambió entre la presentación y el documento, y ninguna cifra tuvo fuente primaria. Su propia recomendación sigue vigente y es la prioridad comercial: **validar la demanda con un piloto académico y métricas de adopción** antes de fijar precios.

6.3 Qué quedó del estudio

RECOMENDACIÓN DEL ESTUDIO	ESTADO
Piloto académico con instituciones	Demo de un solo comando en GHCR y Docker Hub, pensada para aulas y semilleros
Métricas de adopción	Pendiente: la plataforma no registra analítica de uso; se propone un conteo agregado y anónimo de cuentas y macetas activas
Canal directo, comunidad y academia	Portal en smartpot.app, organización pública en GitHub y documentación en español
Modelo <i>freemium</i>	No implementado: hoy todo es gratuito y de código abierto

SECCIÓN 07

Estudio técnico

ASPECTO	PLANTEADO EN LA FORMULACIÓN Y EL ACTA	RESUELTO EN LA PLATAFORMA
Localización	Capas gratuitas en la nube	Un servidor virtual propio con Nginx y TLS; dominios <code>smartpot.app</code> , <code>api</code> , <code>mqtt</code> y <code>mail</code>
Tamaño	Un MVP para dos especies	Ocho contenedores; seis especies; una lectura cada 5 s como máximo por cultivo; hasta 5 macetas virtuales por cuenta
Proceso	ESP32 que envía datos a la API por HTTP	Telemetría y comandos por MQTT v1 con QoS 1, confirmación y última voluntad
Tecnología	Java 17, H2 o MySQL con JPA, React con Vite	Java 21 con Spring Boot 4.1, MongoDB 8, Redis, FastAPI con scikit-learn, React 19 como PWA
Seguridad	JWT, AES y límite de peticiones	Lo anterior más control de dueño, cuenta MQTT por maceta, contenedores endurecidos, CodeQL y SBOM
Insumos para el prototipo físico	Cotización de sensores y actuadores	Sensores de pH y TDS de 25 a 40 USD cada uno, ESP32 de 20 a 30 USD y bombas de 15 a 50 USD: del orden de 85 a 160 USD sin la estructura hidráulica

LECTURA CRÍTICA

El estudio técnico original era una lista de tecnologías más que un estudio de capacidad. La plataforma respondió las preguntas que faltaban (cuánto aguanta, dónde corre, cómo se asegura) con decisiones medibles: límites por cultivo, validadores de datos, pruebas de humo de cada imagen y un endpoint `/health` por servicio.

SECCIÓN 08

Estudio financiero y evaluación

8.1 Presupuestos

DOCUMENTO	MONTO	COMPOSICIÓN
Acta de constitución	1 000 000 COP	40 % infraestructura, 0 % licencias, 50 % tiempo del equipo, 10 % documentación
Restricción del mismo acta	0 a 50 USD	Solo herramientas abiertas y capas gratuitas



DOCUMENTO	MONTO	COMPOSICIÓN
Propuesta de investigación	108 606 000 COP	45 000 000 COP de flujo institucional y 63 606 000 COP de recursos propios; 67,2 millones en equipos

LECTURA CRÍTICA

El acta se contradice: asigna un millón de pesos y a la vez restringe el gasto a 50 USD, unos 200 000 COP. La propuesta de investigación multiplica ese monto por cien y su lista de equipos (cabina para microorganismos, autoclave, incubadora, microscopio) viene de una plantilla de otra disciplina: no corresponde a un sistema hidropónico con ESP32. Un presupuesto de investigación coherente con SmartPot se arma con el prototipo físico (sección 7), el tiempo del equipo y la socialización de resultados.

8.2 Modelo de evaluación

El proyecto nunca tuvo un flujo de caja. Este modelo lo construye con los datos del estudio de mercado y supuestos explícitos, para responder la pregunta que el estudio dejó abierta: ¿qué tendría que pasar para que SmartPot se sostenga?

SUPUESTO	VALOR	ORIGEN
Horizonte	Año 0 de inversión, año 1 académico y gratuito, años 2 y 3 comerciales	Fases de precios del estudio
Inversión	1 000 000 COP	Acta de constitución
Operación del MVP	120 000 COP al año	Tope del estudio de mercado
Operación en escalamiento	20 USD al mes	Tope del estudio de mercado
Tasa de cambio y de descuento	4000 COP por USD; 12 % anual	Supuestos de este análisis
Comisión de pago	5 % del ingreso	Supuesto de este análisis
Costo del equipo	1 300 000 COP al mes: una persona a medio tiempo desde el año 2	Rubro de personal de la propuesta de investigación

ESCENARIO	USUARIOS AÑO 2	CONVERSIÓN A PAGO	PRECIO	SUSCRIPCIONES AÑO 2
Conservador	2500 (+30 % al año 3)	1 %	9 USD	25
Base	3750 (+35 %)	3 %	10,5 USD	112
Optimista	5000 (+40 %)	5 %	12 USD	250



ESCENARIO	VPN SIN COSTO DEL EQUIPO	B/C	VPN CON UNA PERSONA A MEDIO TIEMPO	B/C
Conservador	14,9 millones COP	5,07	-8,6 millones COP	0,68
Base	92,0 millones COP	12,91	68,4 millones COP	3,19
Optimista	242,6 millones COP	16,50	219,1 millones COP	6,59

PRECIO DEL PLAN	PUNTO DE EQUILIBRIO SIN EQUIPO	CON UNA PERSONA A MEDIO TIEMPO
9 USD al mes	2,6 suscripciones	40,6 suscripciones
10,5 USD al mes	2,3 suscripciones	34,8 suscripciones
12 USD al mes	2,0 suscripciones	30,5 suscripciones

LECTURA CRÍTICA

La TIR no aporta aquí: con una inversión de un millón y flujos de decenas de millones da valores de varios cientos por ciento o no existe. Lo que decide la viabilidad es otra cosa:

- La infraestructura casi no pesa: dos o tres suscripciones la pagan.
- El tiempo de las personas sí: sostener a alguien a medio tiempo exige entre 31 y 41 suscripciones, y el escenario conservador no lo logra.
- La variable crítica es la **conversión a pago**, que nadie ha medido. Por eso la prioridad es el piloto con métricas de adopción, no el modelo de precios.

La evaluación social es favorable en cualquier escenario: el software es abierto, la demo no cuesta y el conocimiento queda documentado para otras instituciones.

SECCIÓN 09

Propuesta de investigación

La propuesta formuló SmartPot como investigación aplicada de doce meses: *sistema IoT asistido por IA para la automatización, supervisión remota y análisis predictivo de las condiciones ambientales en cultivos hidropónicos de lechuga*, alineada con los ODS 2, 12 y 13. Su objetivo era pasar de un control reactivo, que actúa cuando la variable ya salió de rango, a uno predictivo.

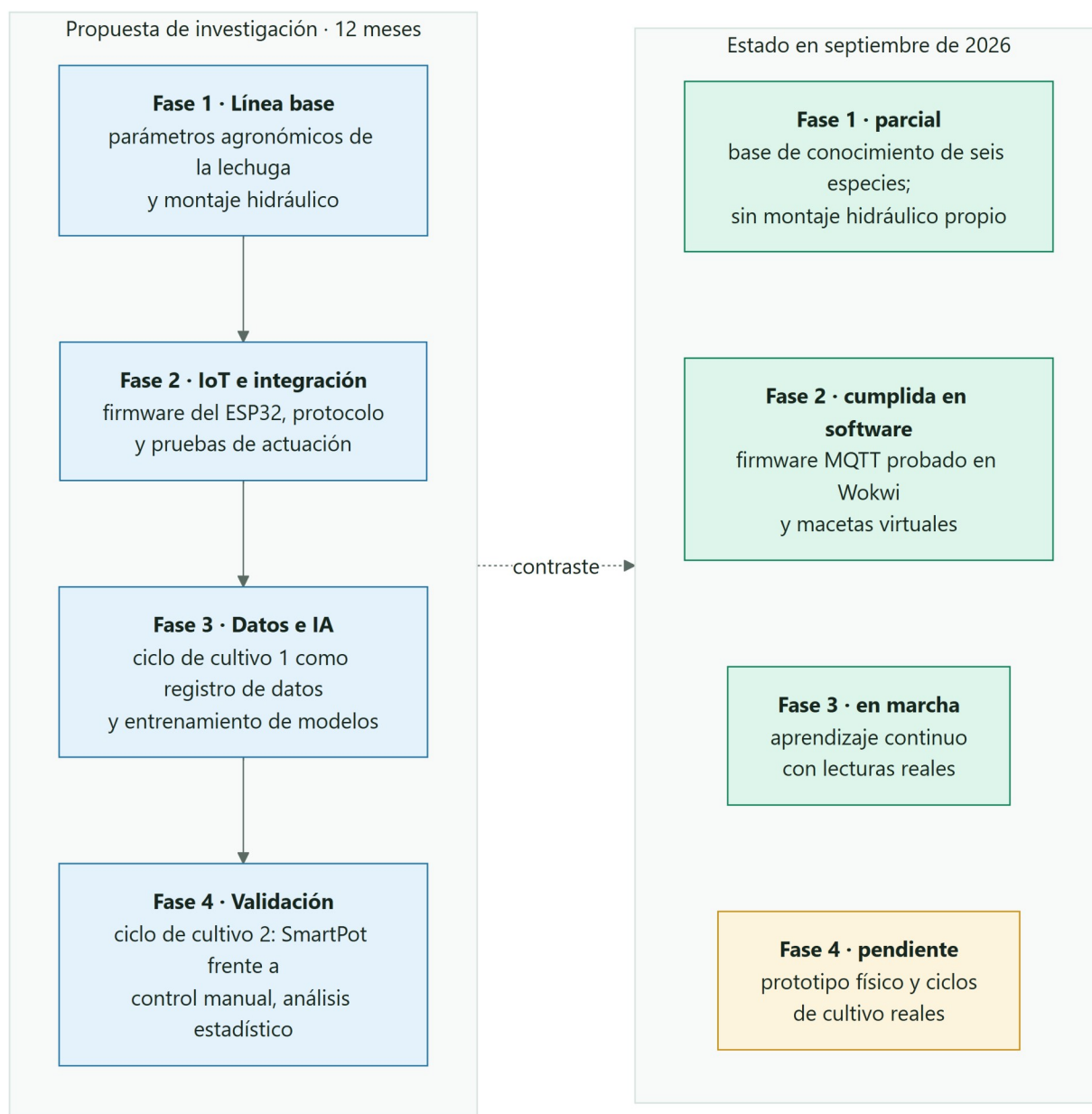


Figura 6 · Fases de la propuesta de investigación y su estado

ELEMENTO	PROPUESTA	LECTURA CRÍTICA
Hipótesis	Implícita: el control asistido por IA mantiene mejor el cultivo que el manual	Conviene escribirla y ligarla a métricas: tiempo en rango, intervenciones, biomasa fresca y litros de agua y solución
Modelos	Regresión y redes LSTM	Una LSTM necesita meses de datos etiquetados; la plataforma empezó con técnicas explicables y aprende con lo que llega (sección 28)



ELEMENTO	PROPUESTA	LECTURA CRÍTICA
Datos	Modo <i>data logger</i> durante un ciclo de 30 a 45 días	Ya existe: la IA guarda las lecturas reales, seudonimizadas, y reentrena por especie
Validación	SmartPot frente a control manual en un segundo ciclo	Pendiente: exige el montaje físico
Productos	Artículo y socialización en un evento	Pendiente; esta documentación es la base del artículo
Bibliografía	Mínimo 20 referencias	Tiene 10; hay que completarla

Referencias de la propuesta, útiles para el trabajo futuro:

REFERENCIA	APORTE
Kurniasari et al. (2025), <i>IOP Conf. Series: Earth and Environmental Science</i>	Hidroponía con ESP32 y monitoreo en tiempo real
Kushawaha et al. (2024), <i>MethodsX</i>	Sistema hidropónico doméstico, pequeño e inteligente
Munaganuri, Yamarthi y Bolem (2025), <i>PeerJ Computer Science</i>	LSTM para agricultura inteligente
Rajaseger et al. (2023), <i>Bioinformation</i>	Tendencias de la hidroponía sostenible
Stevens et al. (2024), <i>PLoS ONE</i>	Manejo de nutrientes de bajo costo en microescala
<i>The Aquaponic Ecosystem Using IoT and IA Solutions (2022), IJWLTT</i>	IoT e IA en acuaponía
Ullah et al. (2019), <i>Intelligent Control and Automation</i>	Monitoreo y control hidropónico de bajo costo
Lowe, Qin y Mao (2022), <i>Water</i>	Aprendizaje automático en tratamiento y monitoreo de agua
Pandi et al. (2024), <i>Earth Science Informatics</i>	Monitoreo ambiental con IA para hidroponía
Mellit et al. (2021), <i>Energies</i>	Monitoreo remoto de invernaderos con IoT y redes profundas



PARTE II

Gestión de proyectos TI

SECCIÓN 10

Acta de constitución: lo que fijó y lo que pasó

DIMENSIÓN	OBJETIVO E INDICADOR DEL ACTA	RESULTADO
Alcance	90 % de las funcionalidades críticas entregadas	13 de 17 requisitos funcionales cumplidos (76 %), 3 parciales y 1 replanteado; 16 de 17 con avance (94 %)
Tiempo	Fase 1 de marzo a abril y fase 2 de abril a julio de 2026, con desviación máxima de una semana	Incumplido: de marzo a junio hubo poca construcción y el grueso llegó en septiembre de 2026
Costo	0 % de sobrecosto	Cumplido en licencias (todo es abierto); el servidor propio no estaba previsto en el acta
Calidad	Cobertura de pruebas de al menos 80 %; menos de 3 defectos críticos al cierre	Sin defectos críticos abiertos al cierre del tercer ciclo; la cobertura no se mide formalmente
Relevancia	Prototipo presentado y aprobado	Plataforma en producción, abierta y con demo pública

PREMISA DEL ACTA	QUÉ PASÓ
Equipo de cuatro personas: gerente, dos desarrolladores y un analista	Cerca de tres de cada cuatro commits humanos los hizo una sola persona
Capas gratuitas de nube como infraestructura	Se abandonaron por un servidor propio: las cuotas y la seguridad no se podían controlar
Hardware físico fuera de alcance	Se respetó; la simulación creció hasta tener macetas virtuales siempre encendidas
Scrum con sprints de dos semanas	La línea base del cronograma usó sprints de tres a cinco días y la construcción se hizo en ciclos (sección 15)

**LECTURA CRÍTICA**

El acta es el documento de gestión más completo del proyecto: objetivos con indicador, riesgos con controles, criterios de aprobación y de cierre, y niveles de autoridad. Su debilidad fue la de muchos proyectos académicos: se escribió para aprobar la planeación y no se volvió a abrir. Dos de sus criterios de cierre se habrían disparado (el alcance creció más del 30 % y el calendario no se cumplió), y ninguno generó una solicitud de cambio.

SECCIÓN 11

Alcance

11.1 Estructura de desglose del trabajo

La EDT se construyó en cuatro niveles; este es su primer nivel y los paquetes de trabajo.

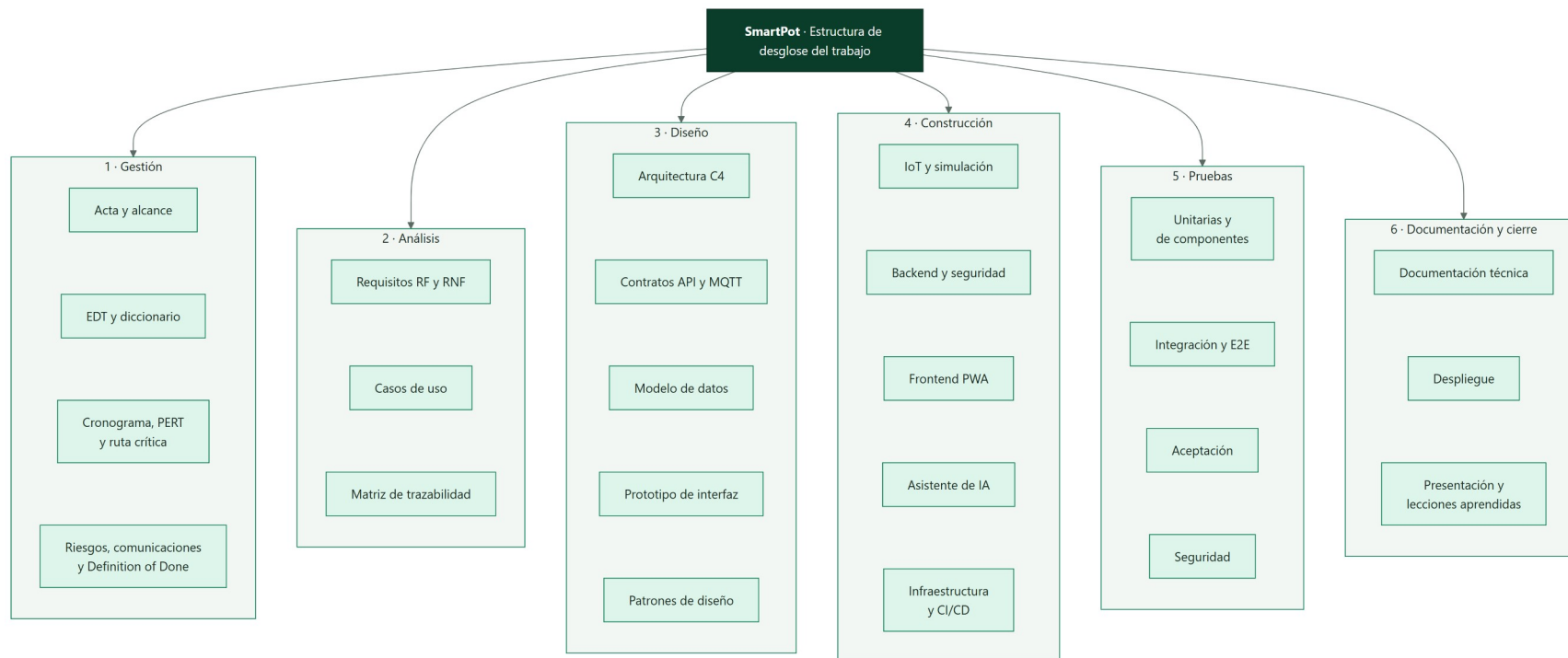


Figura 7 · Estructura de desglose del trabajo



11.2 Priorización y MVP

La matriz de Eisenhower ordenó el trabajo en cuatro grupos y definió el producto mínimo viable.

CUADRANTE	TAREAS	HOY
Hacer	Arquitectura React, Spring Boot y base de datos; simulación básica en Wokwi; primer endpoint que recibe datos	Cumplido y superado
Planificar	Backend completo, frontend, control automático y pruebas de integración	Cumplido
Delegar	Investigación de mercado, documentación de usuario y diseño visual avanzado	Hechos por el equipo; el diseño visual se resolvió con la paleta SmartPot validada
Eliminar	Aplicación móvil, asistentes de voz y cultivos exóticos	La PWA instalable cubre el móvil; la voz sigue fuera; hay seis especies comunes

FUNCIÓN DEL MVP	ESTADO
Monitoreo en tiempo real de pH, temperatura y conductividad	Cumplido con siete variables
Comunicación completa: ESP32, backend, base de datos y frontend	Cumplido con MQTT v1
Control manual de actuadores desde la web	Cumplido, también en bloque

LECTURA CRÍTICA

Eliminar la aplicación móvil fue una buena decisión que la PWA volvió innecesaria de revisar: una sola base de código se instala en el teléfono y en el escritorio. El MVP estaba bien definido, con tres funciones verificables; es el artefacto de la formulación que mejor resistió el paso del tiempo.

SECCIÓN 12

Cronograma

12.1 Red de precedencias

La tabla de precedencias de la formulación codificó las actividades por fase (AN análisis, DI diseño, DE desarrollo, PR pruebas) con su duración y responsable.

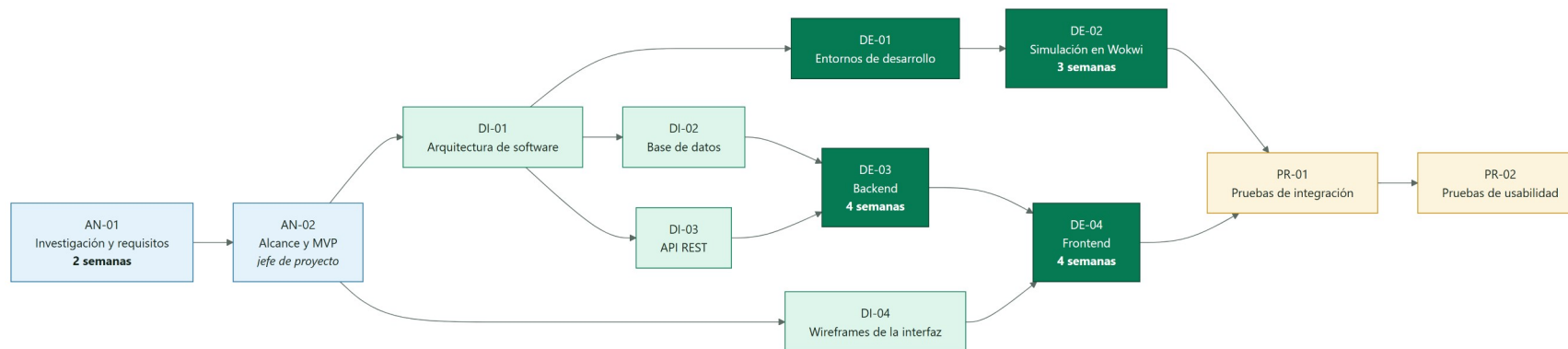


Figura 8 · Red de precedencias de la formulación



LECTURA CRÍTICA

La red hace depender el frontend (DE-04) de un backend terminado (DE-03), lo que alarga la ruta crítica en cuatro semanas. El acta proponía lo contrario en su plan de riesgos: documentar los contratos de la API desde el inicio y usar *mocks* para desacoplar capas. Con los contratos primero, backend y frontend pueden avanzar en paralelo; así se trabajó después con OpenAPI en [/docs](#) y el contrato MQTT v1.

12.2 Ruta crítica

En la gestión del proyecto el cronograma se llevó a MS Project con estimaciones PERT, una EDT de cuatro niveles y el método de la ruta crítica. El informe de tareas críticas lista 57 actividades sin holgura, desde la redacción del acta hasta el archivo del repositorio.

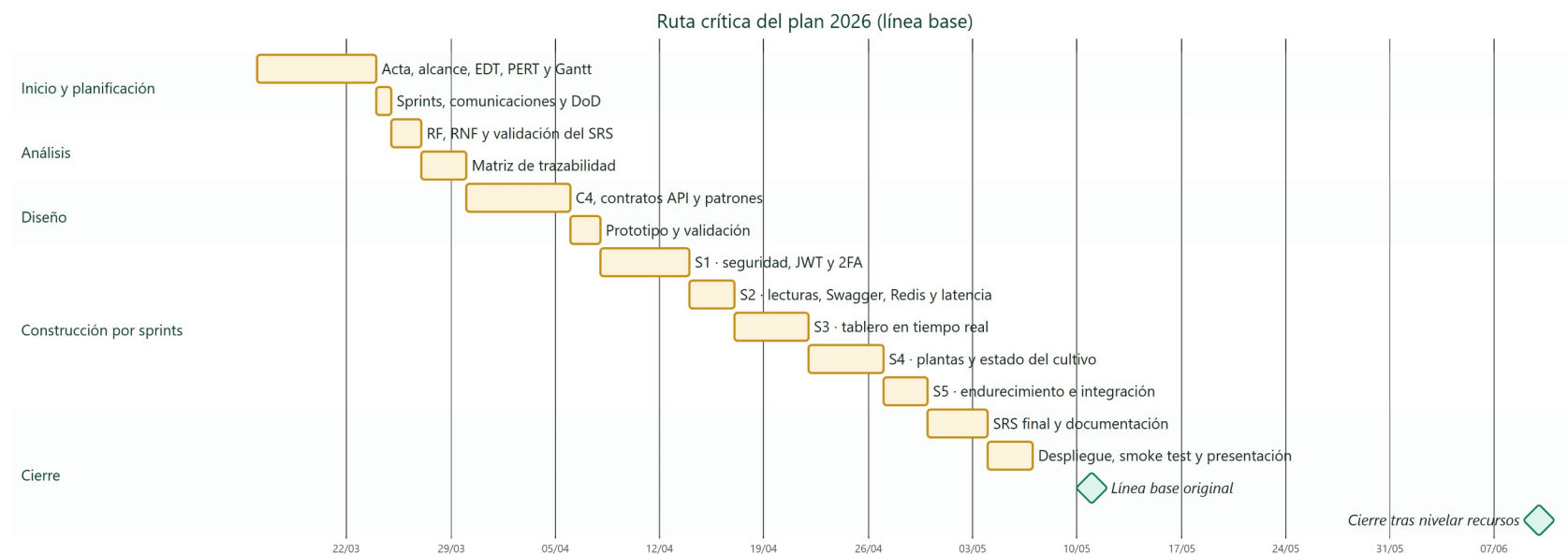


Figura 9 · Ruta crítica de la línea base



VERSIÓN DEL PLAN	FIN DEL PROYECTO	OBSERVACIÓN
Formulación	Diciembre (un semestre)	Cuatro fases mensuales
Hitos del acta	24 de julio de 2026	Ocho hitos entre el 16 de marzo y el 24 de julio
Línea base en MS Project	11 de mayo de 2026	Tareas de uno a dos días y sprints de tres a cinco días
Plan nivelado	10 de junio de 2026	56,06 días de duración tras eliminar sobreasignaciones

LECTURA CRÍTICA

El mismo alcance tuvo tres fechas de fin distintas. La línea base comprimió el plan del acta: terminaba dos meses y medio antes del último hito, con sprints de menos de una semana aunque el acta fijaba dos, y con actividades como «implementar la autenticación en dos pasos» en un día. Además, el informe de tareas críticas muestra todas las actividades al 0 % y en estado «retrasada»: el avance nunca se registró, así que el cronograma no podía avisar de ningún desvío. La lección práctica fue reemplazar el porcentaje declarado por evidencia automática: pruebas y QA en verde por cambio, y despliegues trazables.

SECCIÓN 13

Recursos, asignación y nivelación

TIPO	RECURSOS	CRITERIO DE COSTO
Personas (4)	Gerente de proyecto, desarrollador <i>full stack</i> 1, desarrollador <i>full stack</i> 2, analista y tester	Tasa estándar por hora
Equipos (4)	Un computador por persona	Costo cero: equipos propios
Plataformas (3)	IoT y simulación; backend y base de datos; frontend y despliegue web	Costo total; solo backend y base de datos con 20 USD de contingencia
Herramientas (2)	Repositorio e integración continua; desarrollo y pruebas	Costo cero: gratuitas

TIPO DE ACTIVIDAD	RECURSO BASE	RECURSOS ADICIONALES
Gestión, alcance, riesgos, comunicaciones y cierre	Gerente con su equipo	Repositorio o herramientas solo si la tarea las usa
Requisitos, trazabilidad, validación y pruebas	Analista con su equipo	Herramientas de pruebas y plataformas involucradas
Diseño técnico, backend, seguridad y API	Desarrollador 1	Backend y base de datos, simulación, repositorio
Frontend, tablero e interfaz	Desarrollador 2	Frontend y despliegue web



TIPO DE ACTIVIDAD	RECURSO BASE	RECURSOS ADICIONALES
Simulación IoT e integración con el ESP32 virtual	Desarrollador 1	Simulación y backend

La nivelación fue manual: se movieron horas entre días cercanos hasta que ninguna persona superara 8 horas por jornada, sin sumar recursos que el equipo no tenía. El cierre pasó al 10 de junio de 2026.

TRABAJO DEL INFORME	PROCESO DEL PMBOK (6.ª EDICIÓN)
Ruta crítica	6.5 Desarrollar el cronograma
Hoja de recursos	9.3 Adquirir los recursos (el informe lo rotuló 9.2)
Asignación a actividades	9.2 Estimar los recursos de las actividades
Eliminación de sobreasignaciones	9.6 Controlar los recursos, 6.6 Controlar el cronograma y 4.6 Control integrado de cambios

LECTURA CRÍTICA

El informe de gestión es el mejor razonado de los documentos: explica qué se hizo, cómo y por qué, y acepta que alargar el plazo era más honesto que sostener cargas imposibles. Queda una deuda: el presupuesto del acta dedica la mitad al tiempo del equipo, pero el informe no reporta la línea base de costo que resultó de las tarifas. Cerrar ese círculo permite comparar el costo planeado con el valor ganado.



SECCIÓN 14

Riesgos

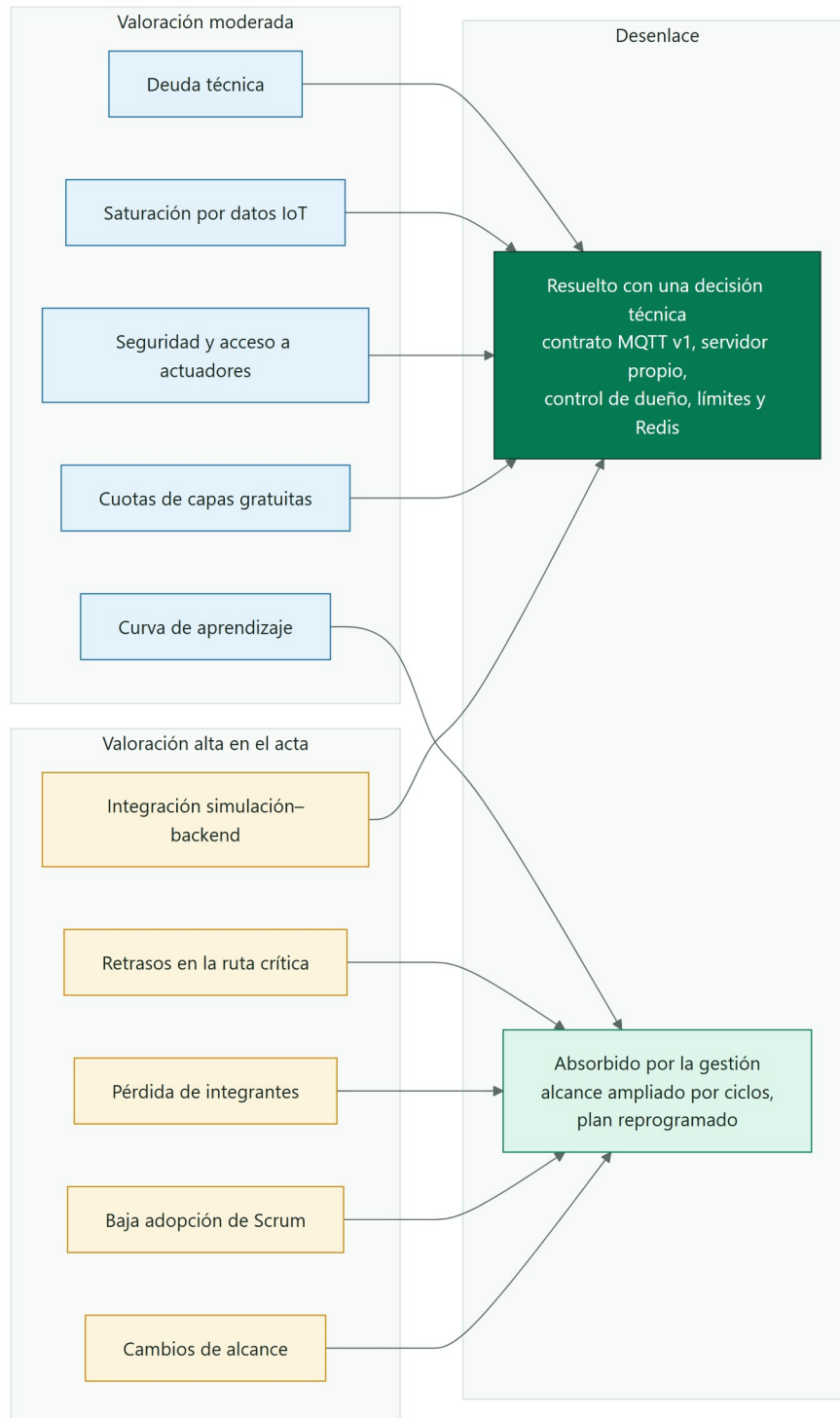


Figura 10 · Riesgos del acta y su desenlace



El desenlace de cada riesgo del acta está en el [recorrido del proyecto](#). Aquí interesa lo que el registro **no** vio:

RIESGO NO IDENTIFICADO	CÓMO APARECIÓ	RESPUESTA
Configuración que rompe producción	Tras cambiar el prefijo a <code>/api/v1</code> , las rutas públicas quedaron con el prefijo viejo y todo respondía 401	Pruebas de controladores con la cadena de seguridad real y un E2E que se registra e ingresa en cada cambio
Servicios expuestos	Base de datos, caché, correo y API publicados en todas las interfaces	Redes internas de Docker, UFW cerrado y solo Nginx expuesto
Acceso a cultivos ajenos	Rutas sin control de dueño	Control de dueño en cada servicio y pruebas que lo verifican
Dependencia de una persona	Tres de cada cuatro commits humanos de una sola persona	Documentación completa, demo reproducible y convenciones iguales en todos los repositorios
Datos personales en documentos	Documentos de trabajo con datos de identificación	La documentación publicada no los incluye

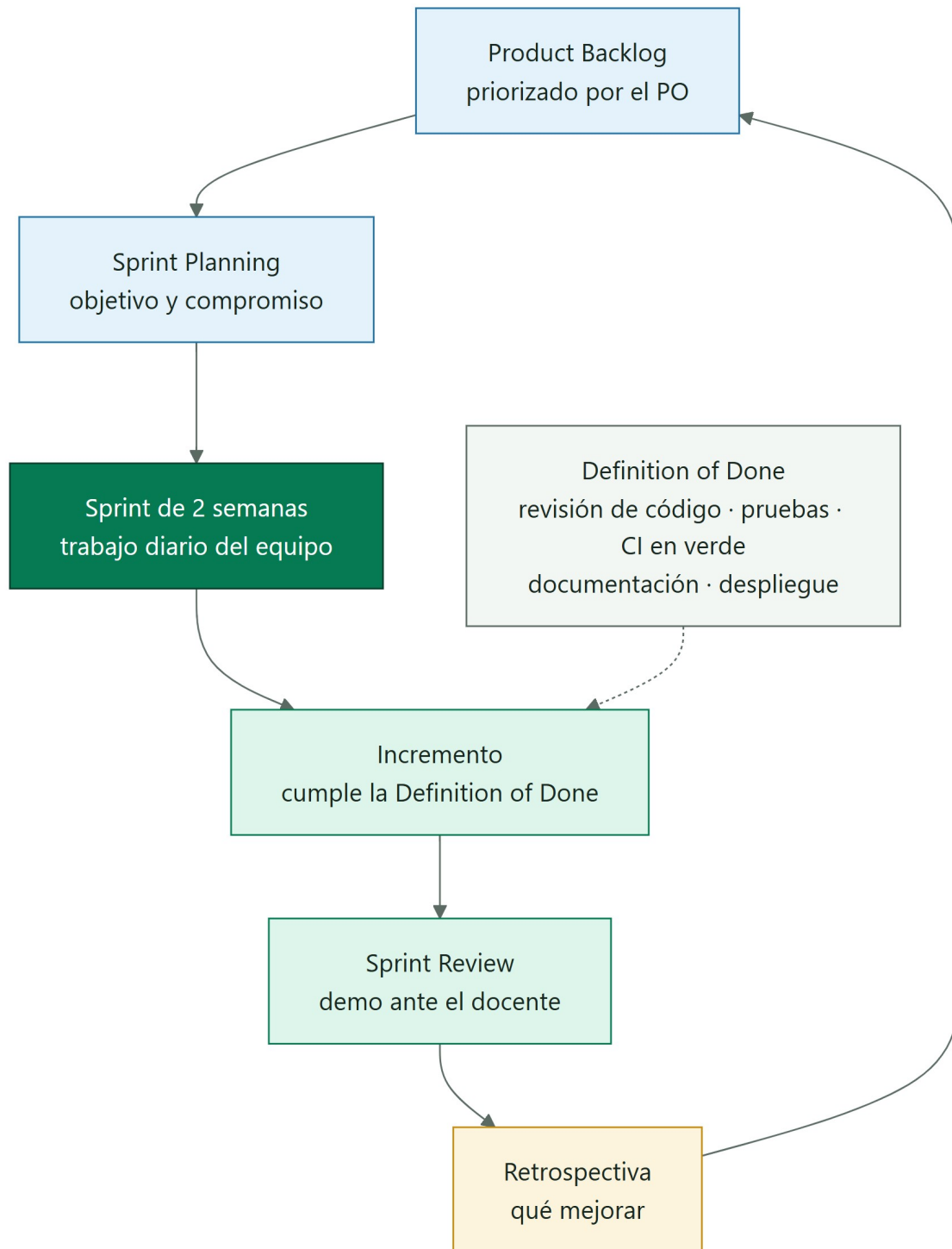
LECTURA CRÍTICA

El registro de riesgos miraba al equipo y al calendario; los riesgos que de verdad se materializaron fueron de configuración y seguridad. Un registro vivo, revisado al final de cada ciclo, los habría anticipado.

SECCIÓN 15

Metodología, comunicaciones y control de cambios

El acta adoptó Scrum con sprints de dos semanas, backlog priorizado por el Product Owner, demos ante el docente y retrospectivas.

**Figura 11** · Ciclo Scrum planeado



PRÁCTICA	PLANEADA	CÓMO SE TRABAJÓ
Iteración	Cinco sprints de dos semanas	Ciclos de mejora con diagnóstico, backlog y criterios de aceptación (parte VII)
Definition of Done	Revisión de código, cobertura mínima y demo	Código con sus pruebas, CI del repositorio en verde, QA de extremo a extremo en verde, documentación al día y despliegue desde GHCR con <code>/health</code> en UP
Comunicaciones	Canales, frecuencia y demos	Commits pequeños en inglés con un solo propósito, READMEs por repositorio y estos documentos
Control de cambios	Proceso formal y backlog congelado por sprint	Cada ciclo abre un alcance nuevo; el contrato MQTT versionado (v1) protege a las macetas de los cambios
Métricas	Velocidad y <i>burndown</i>	Pruebas, estado del QA y despliegues por ciclo (sección 38)

LECTURA CRÍTICA

Scrum se planeó completo pero no se ejecutó como tal: con una persona haciendo la mayor parte del trabajo, las ceremonias pierden sentido. Lo que sí funcionó fue su esencia: iteraciones cortas, un incremento desplegable al final de cada una y una definición de terminado automatizada. Para un equipo pequeño, un tablero Kanban con límites de trabajo en curso y la misma definición de terminado es más honesto que un Scrum nominal.

PARTE III

Análisis

SECCIÓN 16

De la necesidad al requisito

Los requisitos tuvieron tres versiones:

VERSIÓN	FECHA	CONTENIDO
Matriz de requisitos	Agosto de 2024	Requisitos no funcionales RNF001 a RNF007 en versión 0.1, todos «sin empezar»; el contrato de la API era el RNF007
Acta de constitución	Marzo de 2026	RF-001 a RF-017 y RNF-001 a RNF-006; el contrato de la API pasó a ser el RF-015
Plataforma	Septiembre de 2026	13 requisitos funcionales cumplidos, 3 parciales y 1 replanteado; 4 no funcionales cumplidos y 2 pendientes

El estado de cada requisito está en el [recorrido del proyecto](#). Esta sección revisa su **calidad** con las características de un buen requisito (necesario, sin ambigüedad, verificable y completo).

REQUISITO	PROBLEMA	REESCRITURA VERIFICABLE
RF-002 Calibrar sensores	La descripción quedó cortada a mitad de frase	El sistema permite ajustar la escala de cada sensor de un cultivo y registra quién la cambió
RF-009, RF-012, RF-013 y RF-014	Usan «podrá» o «permitirá» junto a otros con «debe»: no queda claro si son obligatorios	Todos con «debe» y una prioridad MoSCoW explícita
RF-014 Gestión de nutrientes	«De manera efectiva» no se puede probar	El sistema bloquea la dosificación de nutrientes cuando el TDS supera el máximo de la especie
RF-017 Estado general	Tres niveles fijos	Índice de 0 a 100 con cinco niveles y el aporte de cada variable (implementado así)
RNF-001 Doble verificación	Se justificaba con ISO/IEC 25000, que es un marco de calidad y no exige 2FA	La cuenta admite un segundo factor por aplicación de autenticación; motivo: la característica de seguridad de ISO/IEC 25010
RNF-002 Paleta	«Un diseño llamativo que garantice la permanencia» no se puede verificar	Los textos cumplen contraste WCAG AA y las series de las gráficas son distinguibles con daltonismo (lo segundo ya se valida)



REQUISITO	PROBLEMA	REESCRITURA VERIFICABLE
RNF-003 Tiempo real	En la matriz de 2024 hablaba de «canciones guardadas»: un error de copia de otro proyecto	La PWA muestra una lectura nueva en menos de 30 s desde que la maceta la publica
RNF-005 Navegación clara	«Entendible para todas las edades» no es medible	Toda función principal está a dos toques o menos desde el panel

LECTURA CRÍTICA

Faltaron requisitos no funcionales que la plataforma tuvo que resolver igual: disponibilidad, rendimiento (la latencia de 500 ms solo aparecía en los criterios de aprobación), privacidad y retención de datos, observabilidad y mantenibilidad. Hoy existen como decisiones de construcción (retención de un año, límites de peticiones, `/health` por servicio, CI en todos los repositorios) y conviene elevarlos a requisitos para poder probarlos.

SECCIÓN 17

Casos de uso

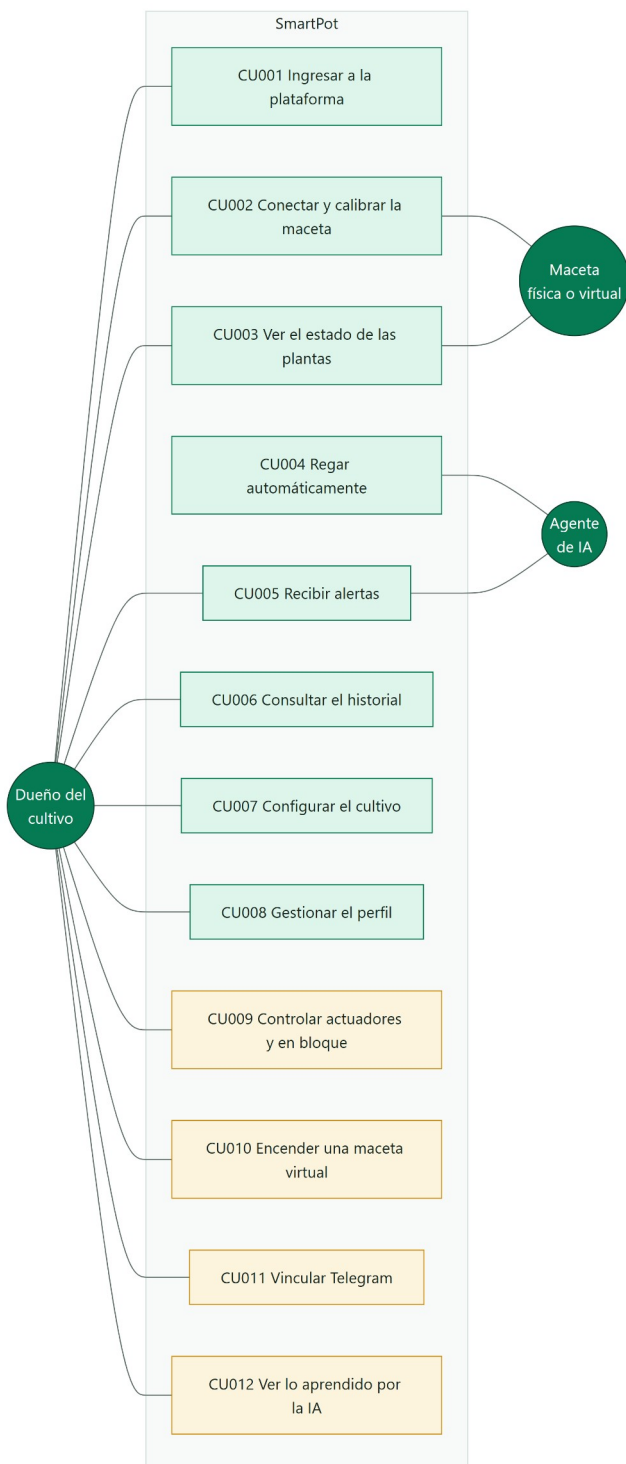


Figura 12 · Casos de uso de SmartPot



Los ocho casos de uso del diseño se conservan y la construcción sumó cuatro: controlar en bloque, encender una maceta virtual, vincular Telegram y ver lo aprendido por la IA. El cambio de fondo es un actor nuevo: el **agente de IA** aparece como actor secundario de «regar automáticamente» y «recibir alertas». En el diseño la maceta decidía; en la plataforma decide un agente que la persona activa y puede auditar.

SECCIÓN 18

Trazabilidad

El acta pedía una matriz de trazabilidad de requisito a módulo y a caso de prueba. Esta es la matriz con la plataforma actual:

REQUISITO	CASO DE USO	DÓNDE VIVE	CÓMO SE PRUEBA
RF-001 Acceder	CU001	API <code>security</code> y <code>users</code> ; PWA de ingreso	E2E: registro, ingreso y rechazo sin sesión
RF-002 Calibrar sensores	CU002	Firmware <code>ADCSensor</code> ; PWA del dispositivo	Pruebas del firmware
RF-003 Estado de las plantas	CU003	API <code>overview</code> y <code>crops</code> ; IA de salud	E2E: panel general
RF-004 Riego automático	CU004	API <code>ai</code> (agente de automatización); IA del agente	Pruebas del agente y de sus reglas
RF-005 Alertas	CU005	API <code>notifications</code> y <code>channels</code>	Pruebas de notificaciones y del reenvío a Telegram
RF-006 Historial	CU006	API <code>readings</code> ; PWA de historial y CSV	E2E: series agregadas
RF-007 Registrar lecturas	CU003	API <code>mqtt</code> y <code>readings</code> , con retención de un año	E2E: lectura publicada por MQTT
RF-008 Configurar parámetros	CU007	Base de conocimiento de la IA	Pruebas de los perfiles por especie
RF-009 Control de luz	CU004	Reglas del agente con descanso nocturno	Pruebas de reglas
RF-010 Control manual	CU009	API <code>commands</code> y <code>actuators</code>	E2E: comando con ACK y orden en bloque
RF-011 Perfil	CU008	API <code>users</code> y <code>channels</code>	Pruebas de controladores
RF-012 Reportes	CU006, CU012	Resumen, CSV, análisis de flota y aprendizaje	Pruebas de la IA; E2E de aprendizaje
RF-013 Gestión de plantas	CU007	API <code>crops</code> y <code>actuators</code>	Pruebas de controladores



REQUISITO	CASO DE USO	DÓNDE VIVE	CÓMO SE PRUEBA
RF-014 Nutrientes	CU009	Dosificadores y regla de bloqueo de nutrientes	Pruebas de reglas
RF-015 Conectar con la API	CU002	Contrato MQTT v1 y OpenAPI en /docs	Pruebas de contrato y E2E
RF-016 Gráficos históricos	CU006	PWA: gráfico con banda ideal y comparador	Pruebas de componentes
RF-017 Estado general	CU003	IA: índice difuso de salud	Pruebas de la IA

SECCIÓN 19

Reglas de negocio y base de conocimiento

REGLA	VALOR	POR QUÉ
Frecuencia máxima de lecturas	Una cada 5 s por cultivo	Proteger la base de datos y la API de una maceta mal configurada
Retención de lecturas	Un año	Historial útil sin crecimiento indefinido
Evaluación del agente	Cada 5 minutos, con las últimas 48 lecturas	Suficiente para ver tendencias sin saturar la IA
Enfriamiento por actuador	10 minutos	Evitar que el agente encienda y apague en ciclo
Vencimiento de un comando	2 minutos sin confirmación	Avisar a la persona si la maceta no responde
Sensor sospechoso	El agente no actúa	Ante un valor imposible es más seguro no regar
Límite de peticiones	300 por minuto; 10 por minuto en autenticación	Frenar abuso y fuerza bruta
Recuperación de contraseña	Enlace de un solo uso que vence en 30 minutos	No enviar credenciales por correo
Macetas virtuales	Hasta 5 por cuenta	Mantener acotado el simulador compartido
Descanso nocturno de la luz	De 22:00 a 6:00, hora de Colombia	Respetar el fotoperiodo

La base de conocimiento nació de la investigación temática del diseño (lechuga y tomate) y hoy cubre seis especies: lechuga, tomate, fresa, albahaca, espinaca y pimentón. El detalle de sus rangos está en el [recorrido del proyecto](#) y en la [documentación técnica](#).



PARTE IV

Diseño

SECCIÓN 20

Arquitectura

20.1 Contexto

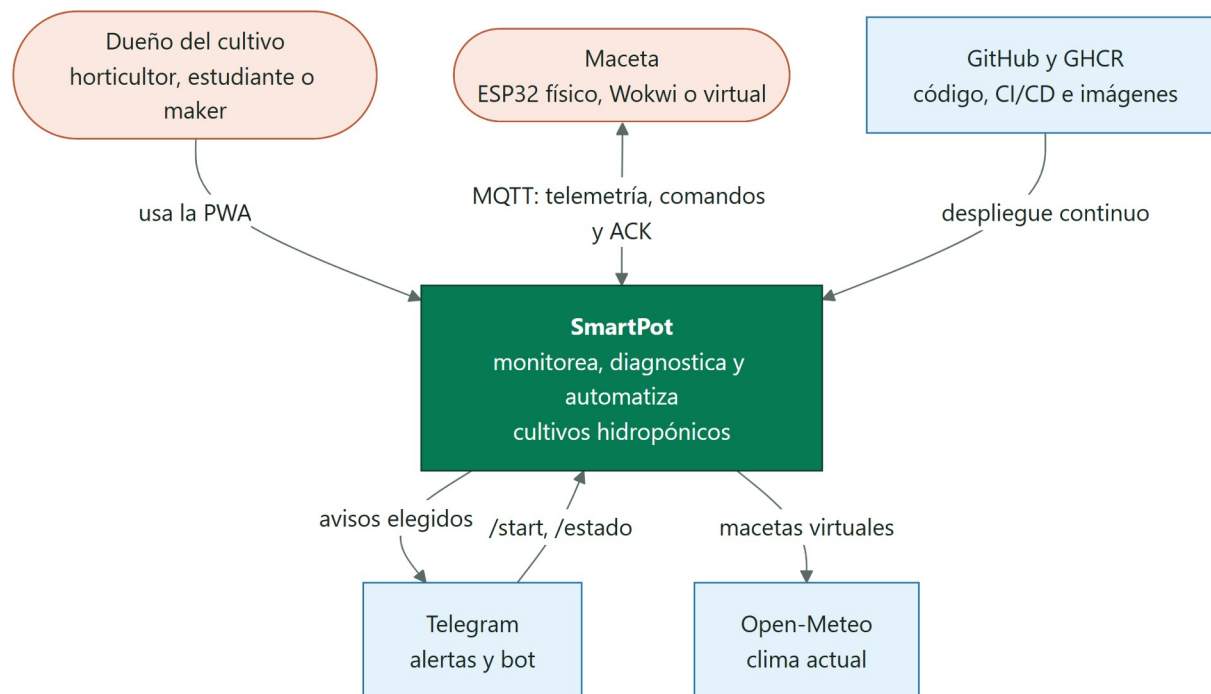
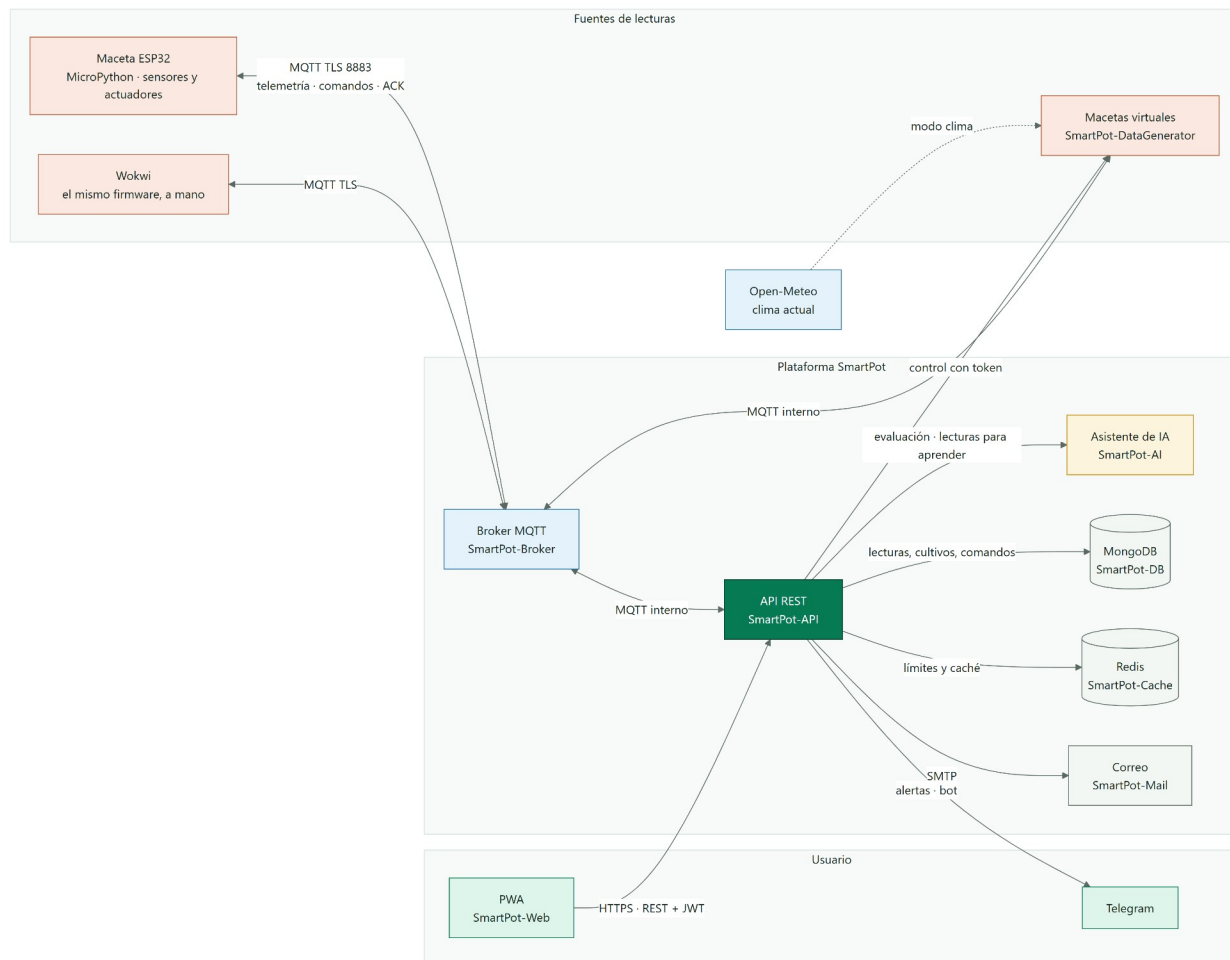


Figura 13 · Contexto del sistema



DECISIÓN	ALTERNATIVA DESCARTADA	POR QUÉ	CONSECUENCIA
MQTT v1 entre maceta y plataforma	Consulta periódica por HTTP	Conexión liviana, bidireccional y con estado en línea (última voluntad)	Un broker más que operar y asegurar
MongoDB con validadores	H2 o MySQL con JPA	Lecturas con variables opcionales y consultas por cultivo y fecha	La integridad se protege con \$jsonSchema e índices
IA como servicio aparte en Python	Reglas dentro de la API en Java	El ecosistema de ciencia de datos está en Python	Contrato HTTP interno con token entre API e IA
El agente decide en la plataforma	La maceta decide sola	Una maceta barata se beneficia de todo lo aprendido	Sin conexión, la maceta solo mide y muestra en su pantalla



DECISIÓN	ALTERNATIVA DESCARTADA	POR QUÉ	CONSECUENCIA
Simulador como servicio	Solo Wokwi	Demo y QA que no dependen de una pestaña abierta	Un contenedor más, interno y con token
Servidor propio con Docker	Capas gratuitas en la nube	Sin cuotas y con control total de la seguridad	La operación queda en manos del equipo
PWA	Aplicación móvil nativa	Un solo código para teléfono y escritorio	Sin acceso a funciones nativas avanzadas, que no se necesitan
Telegram en la API	Bot dentro del ESP32	Permisos y datos de cada cuenta en un solo lugar	La maceta no conoce ningún canal
Despliegue central	Cada repositorio despliega	Un solo lugar con acceso al servidor	Los servicios piden el despliegue y el central lo encola

SECCIÓN 21

Del diseño original al actual

Los diagramas de la fase de diseño describían otra aplicación. Ponerlos lado a lado muestra qué ideas sobrevivieron y cuáles se movieron de lugar.



21.1 Clases

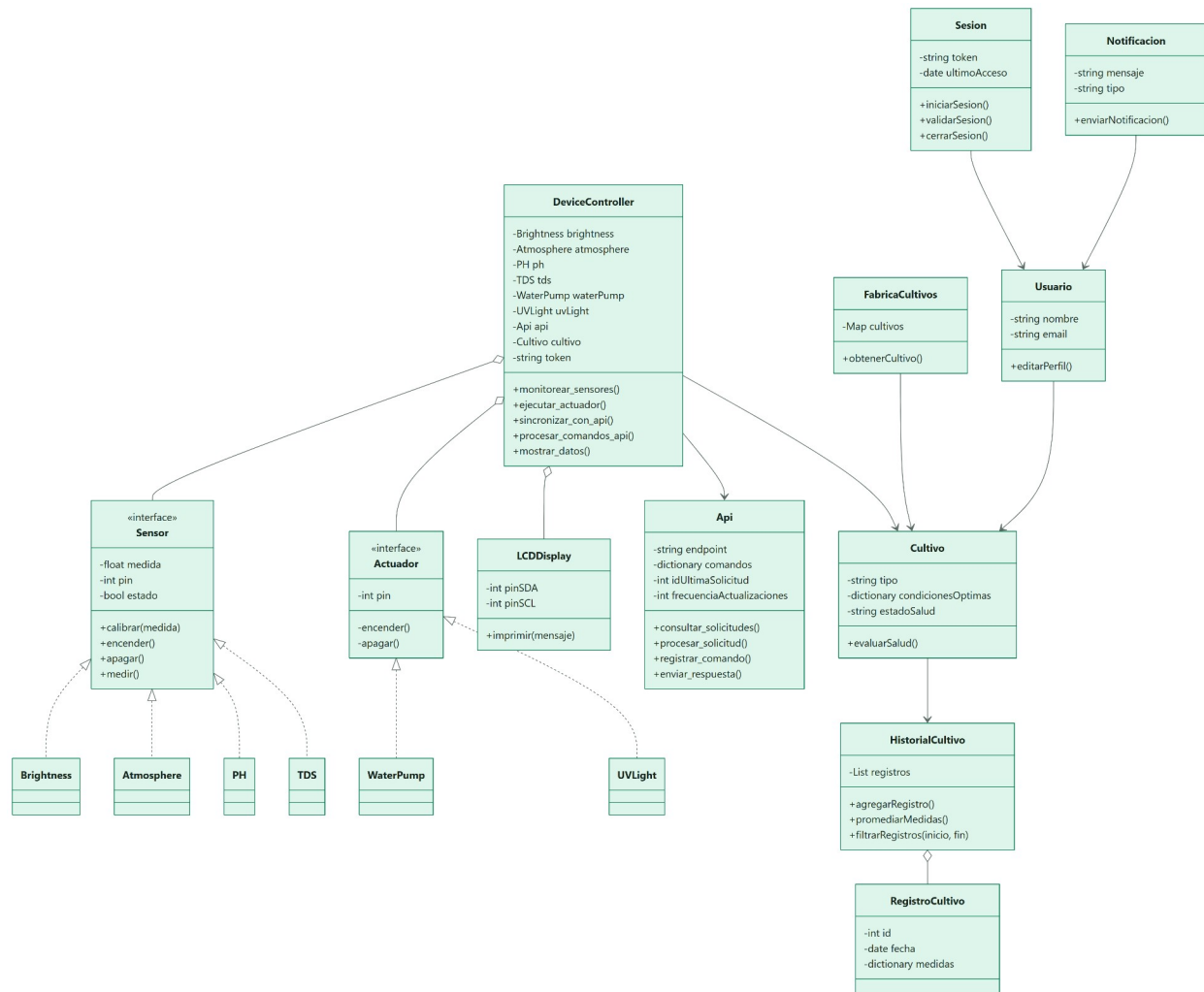


Figura 15 · Clases del diseño original

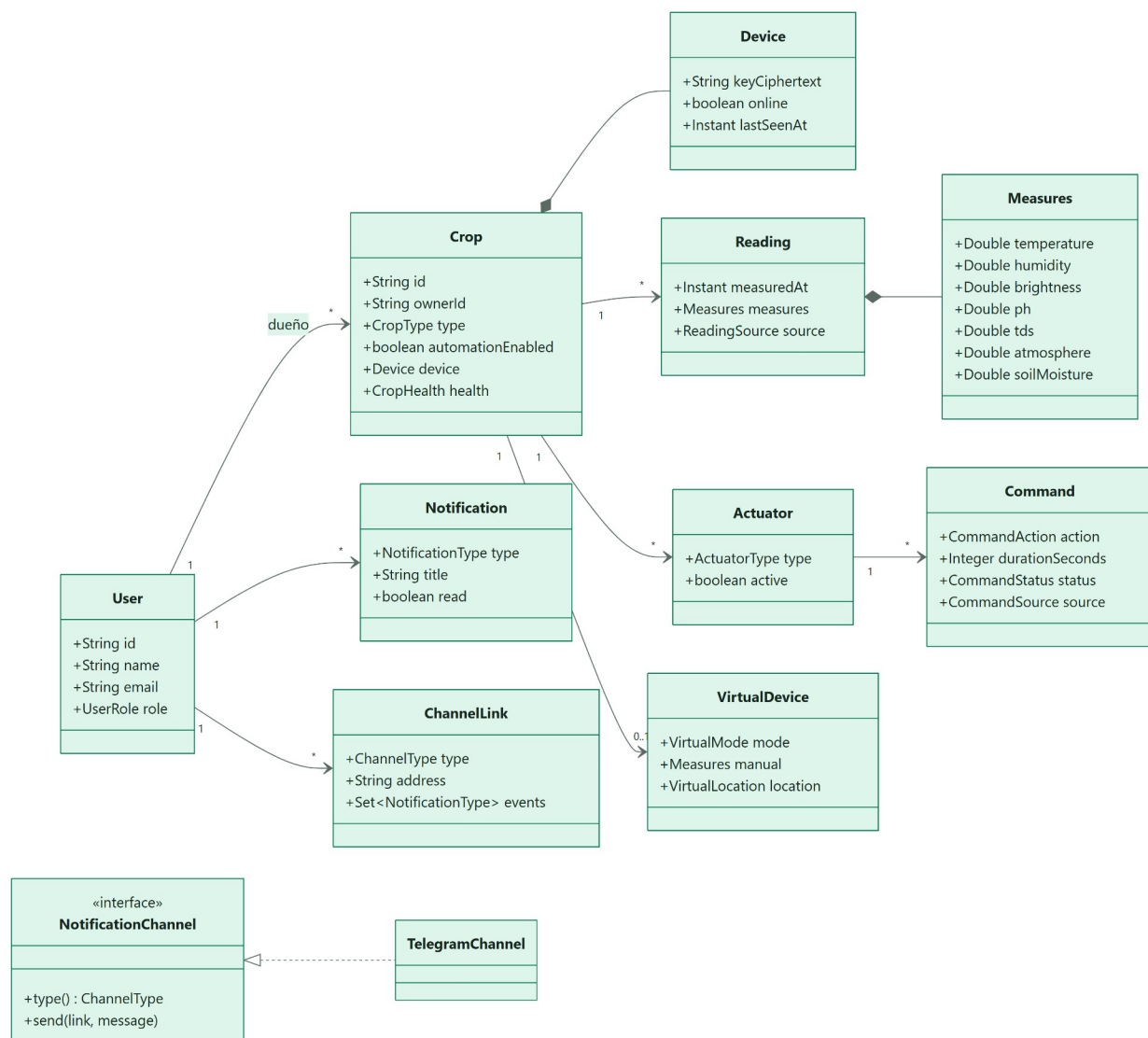


Figura 16 · Clases del dominio actual

DISEÑO ORIGINAL	ACTUAL	LECTURA
DeviceController con sensores, actuadores, API y cultivo	La maceta solo mide y obedece; el dominio vive en la API	La inteligencia se movió a la plataforma
Cultivo.evaluarSalud()	Índice de salud, diagnóstico y pronóstico en la IA	Un método se volvió un servicio
FabricaCultivos con un mapa de cultivos	Perfiles por especie compartidos	La fábrica era, en realidad, datos
Sesion con token y último acceso	JWT sin estado	No hace falta guardar sesiones
Api con consulta de solicitudes	Comandos con id, estados y confirmación	El «identificador de la última solicitud» se volvió una máquina de estados

21.2 Componentes

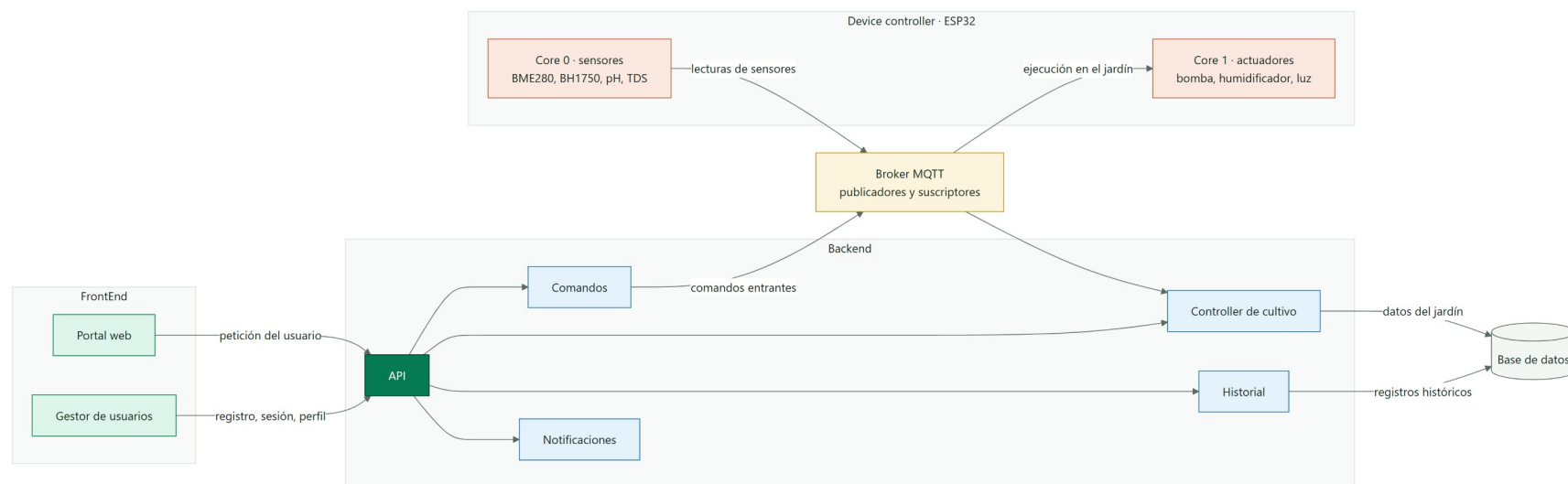


Figura 17 · Componentes del diseño original

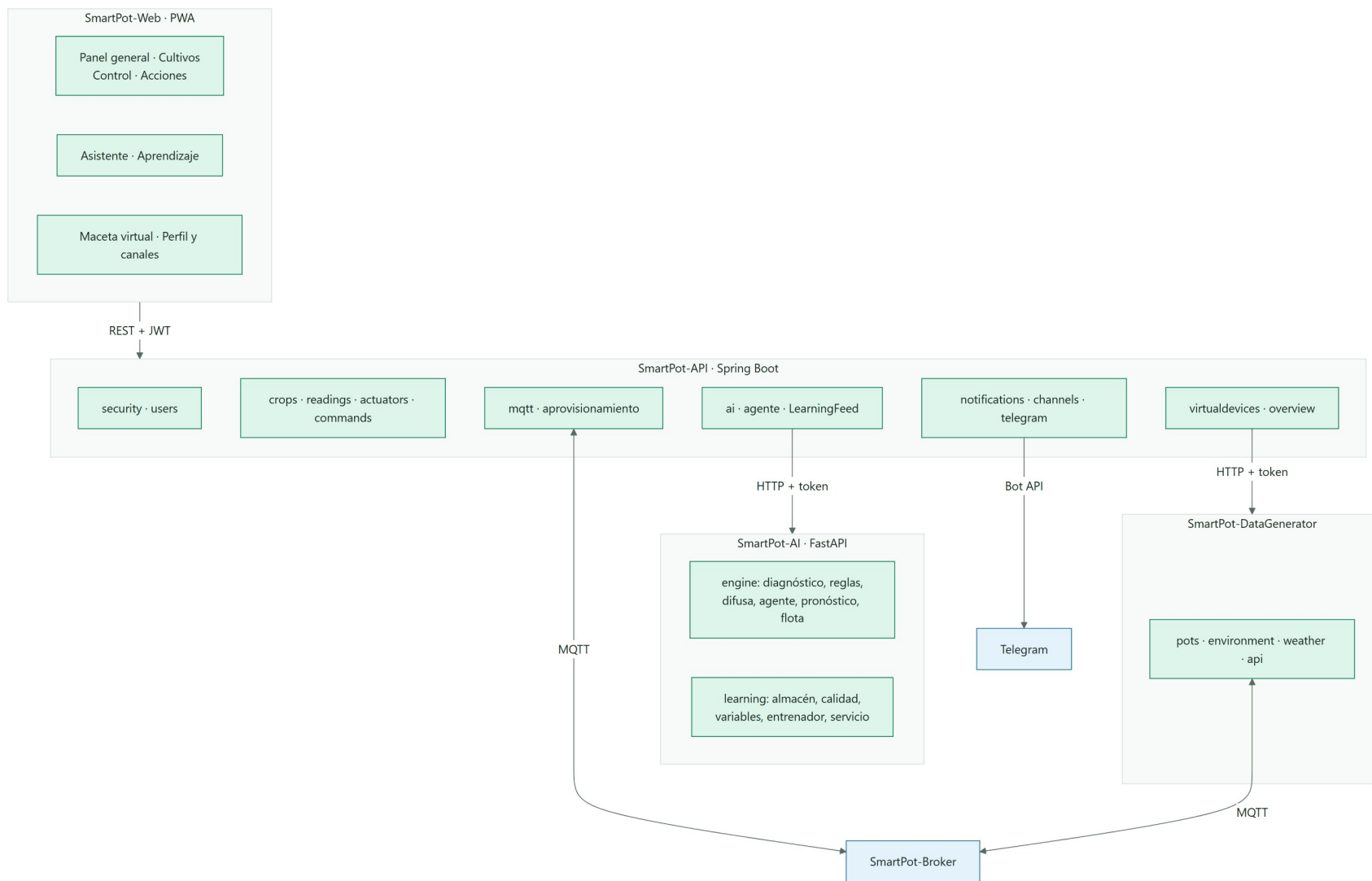


Figura 18 · Componentes y paquetes actuales

El diseño original ya tenía un broker MQTT y dividía el ESP32 en dos núcleos (sensores y actuadores). Se conservó la idea del broker y se simplificó el firmware a un ciclo principal; lo que cambió fue el backend, que pasó de cuatro componentes genéricos a dominios con la misma organización por capas.

21.3 Comportamiento: enviar un comando

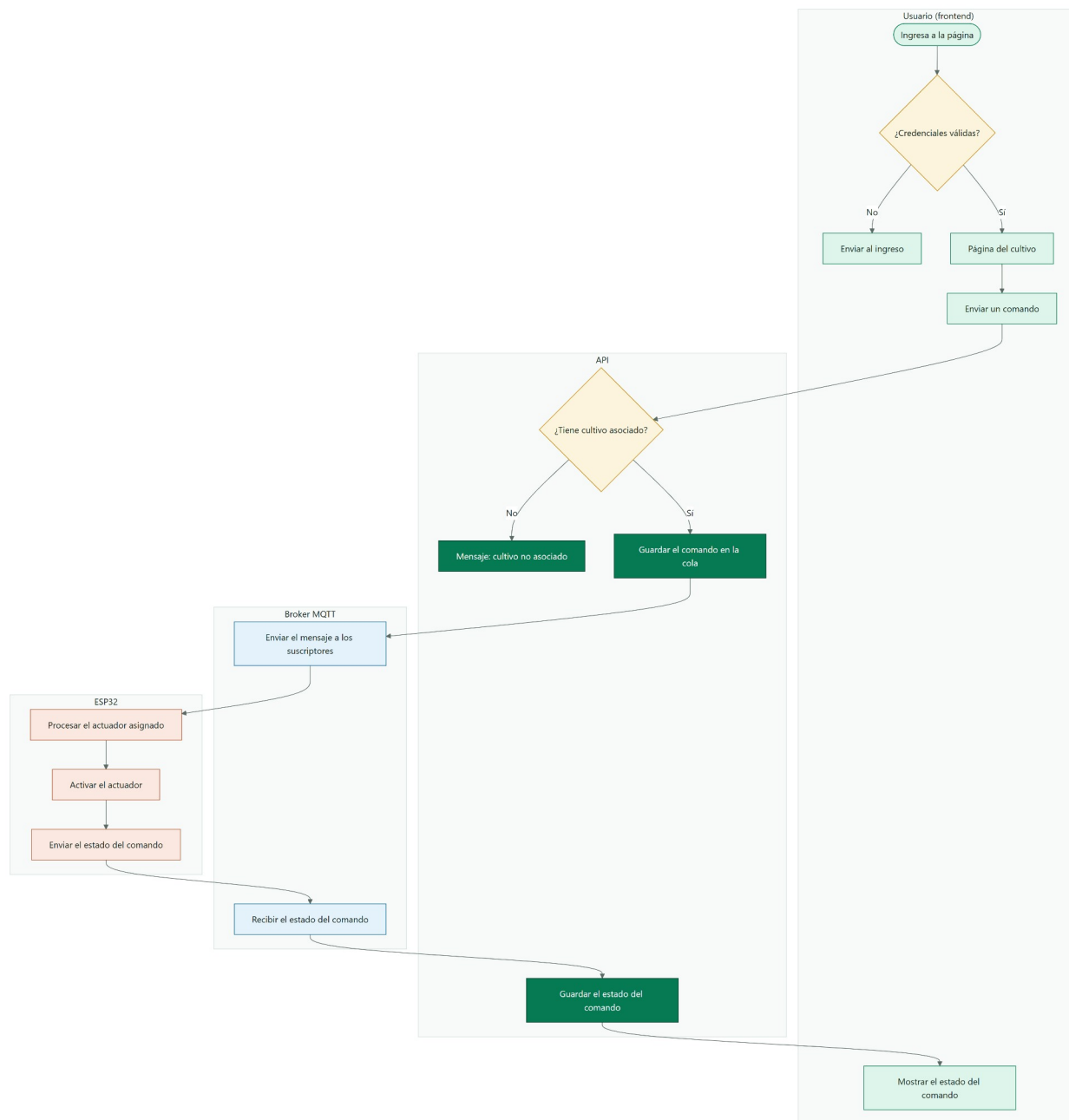


Figura 19 · Actividad del diseño original: enviar un comando

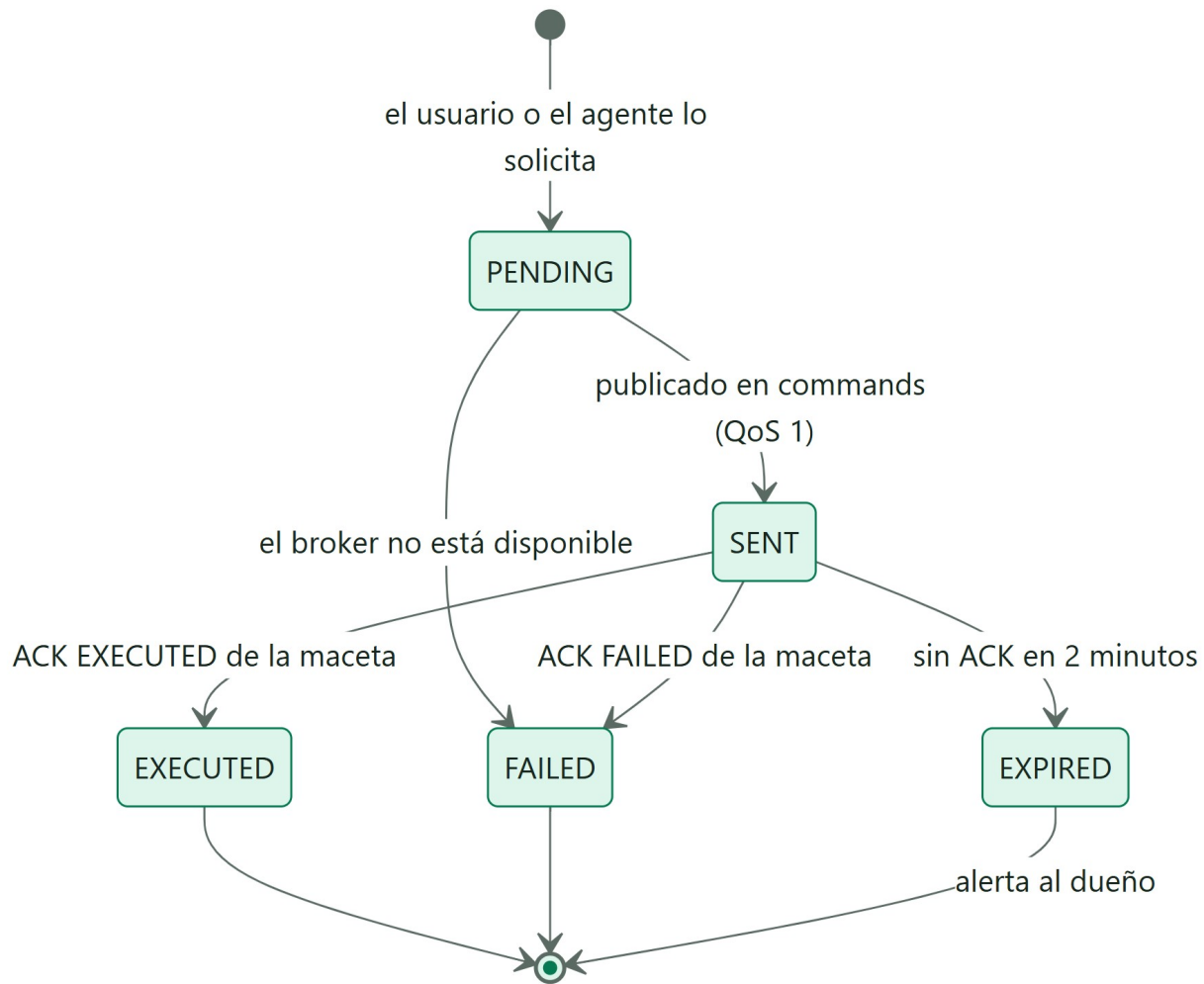


Figura 20 · Estados de un comando hoy

El flujo del diseño se mantuvo casi igual: validar sesión, verificar el cultivo, encolar, publicar y esperar el estado. Lo nuevo es que el estado es explícito (PENDING, SENT, EXECUTED, FAILED, EXPIRED) y que un comando sin respuesta vence en dos minutos y avisa.

21.4 Comportamiento: datos históricos y una lectura

La secuencia original se centraba en consultar el historial; la actual muestra el camino completo de una lectura, desde la maceta hasta la acción automática, porque ese es hoy el flujo crítico del sistema.

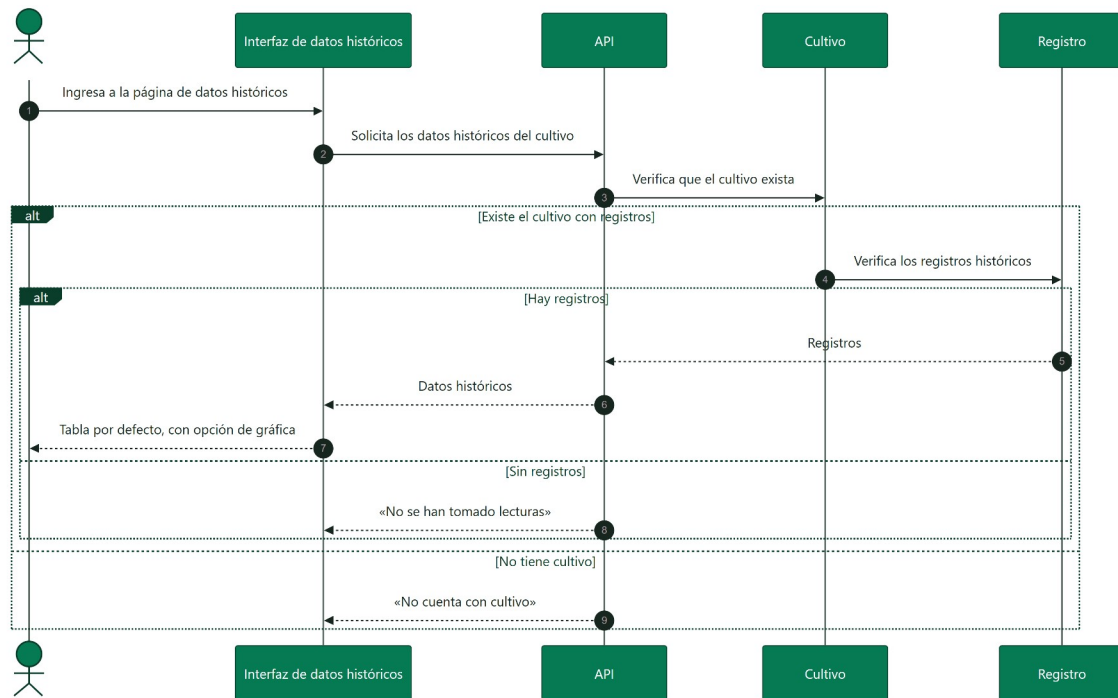


Figura 21 · Secuencia del diseño original: datos históricos

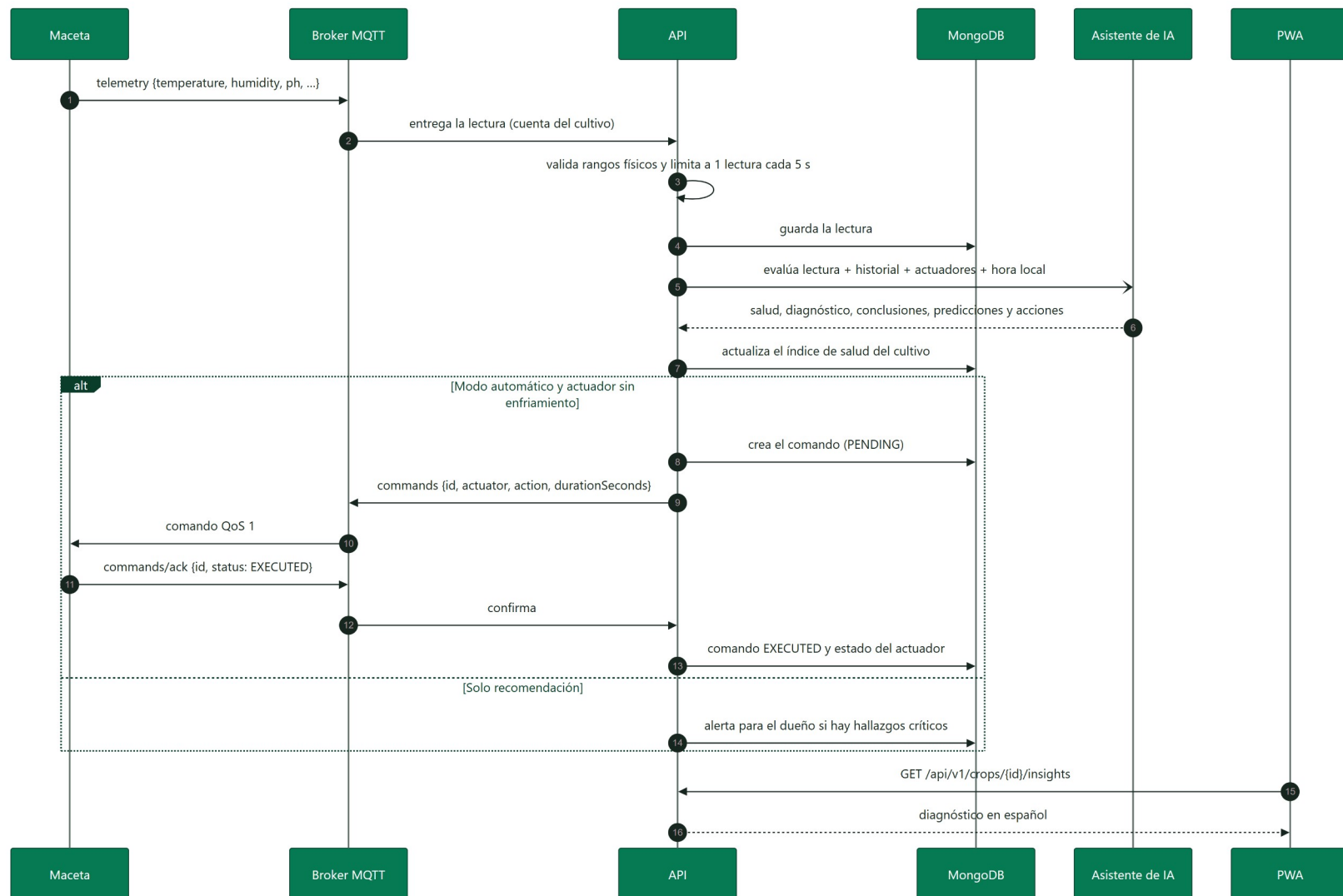


Figura 22 · Secuencia de una lectura con automatización



21.5 Actividad: control automático

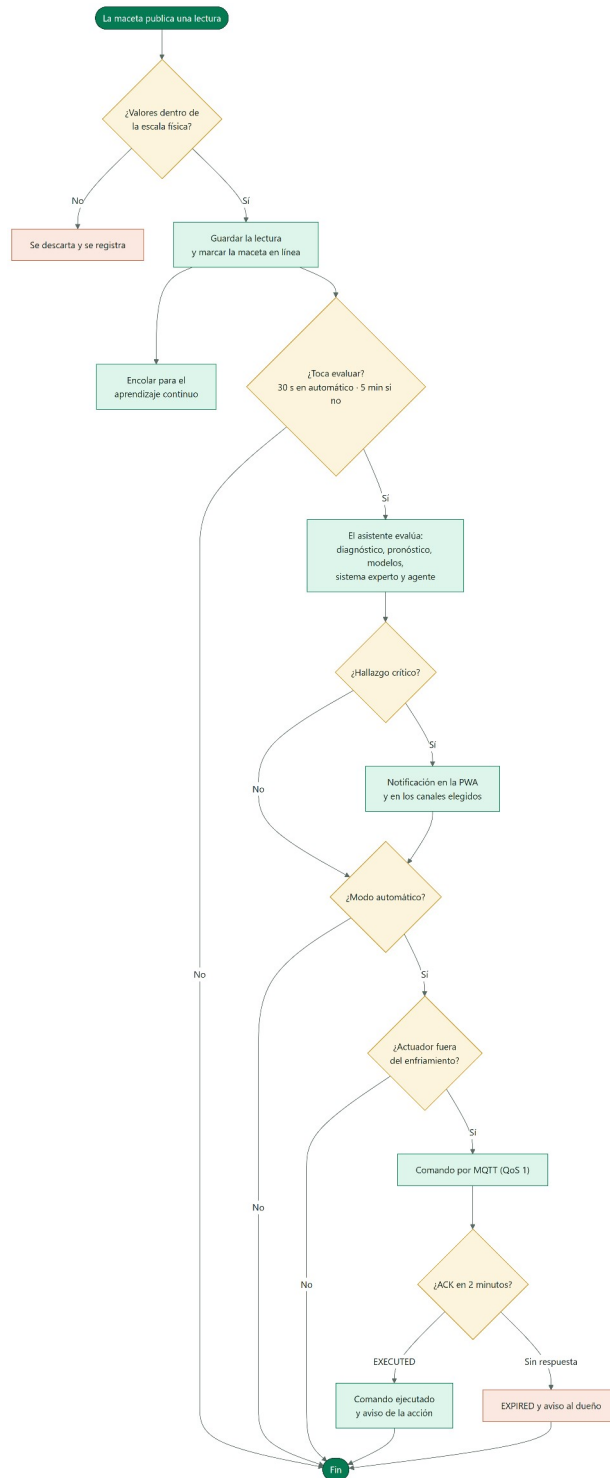


Figura 23 · Actividad del control automático

21.6 Datos

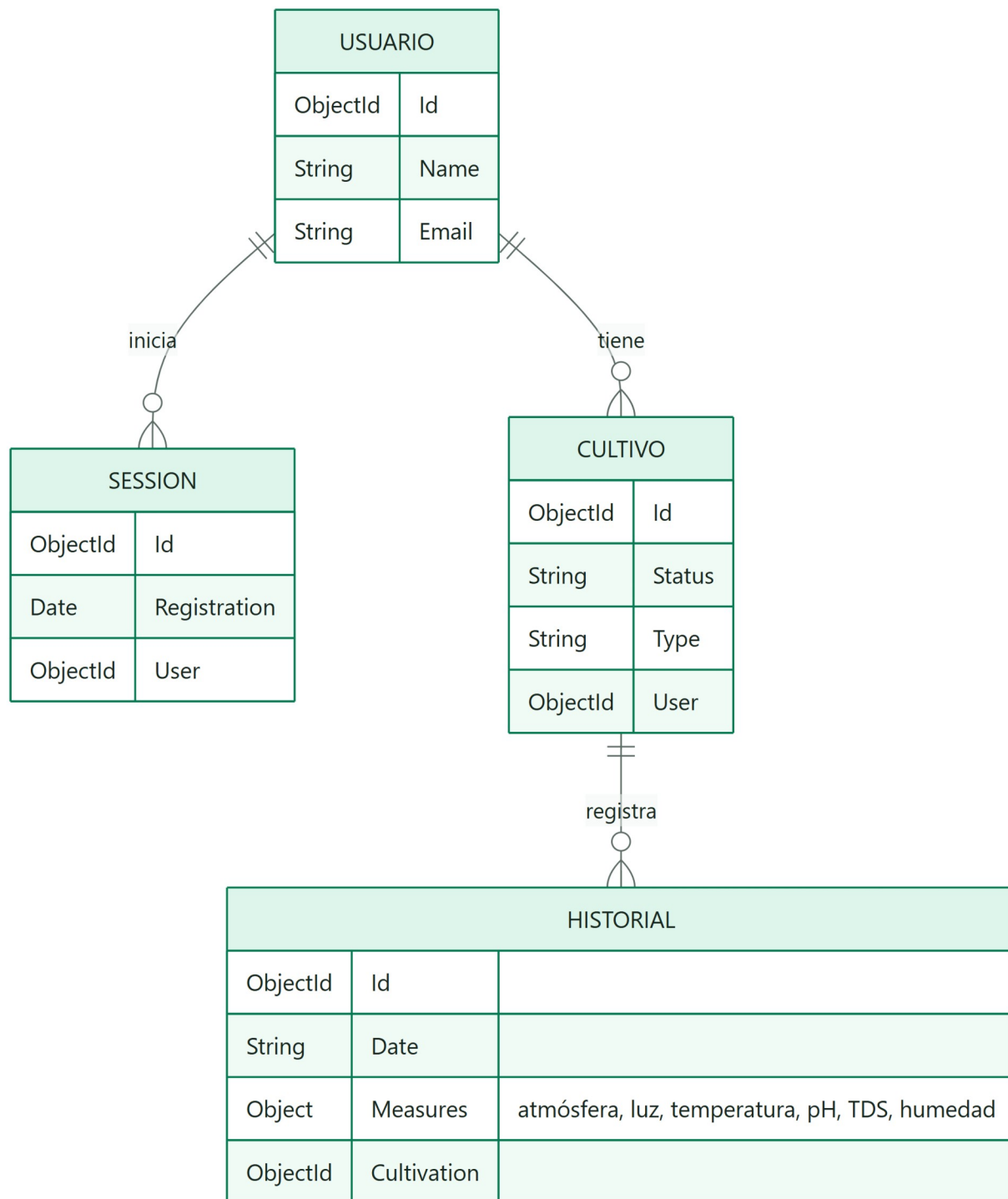


Figura 24 · Modelo de datos del diseño original

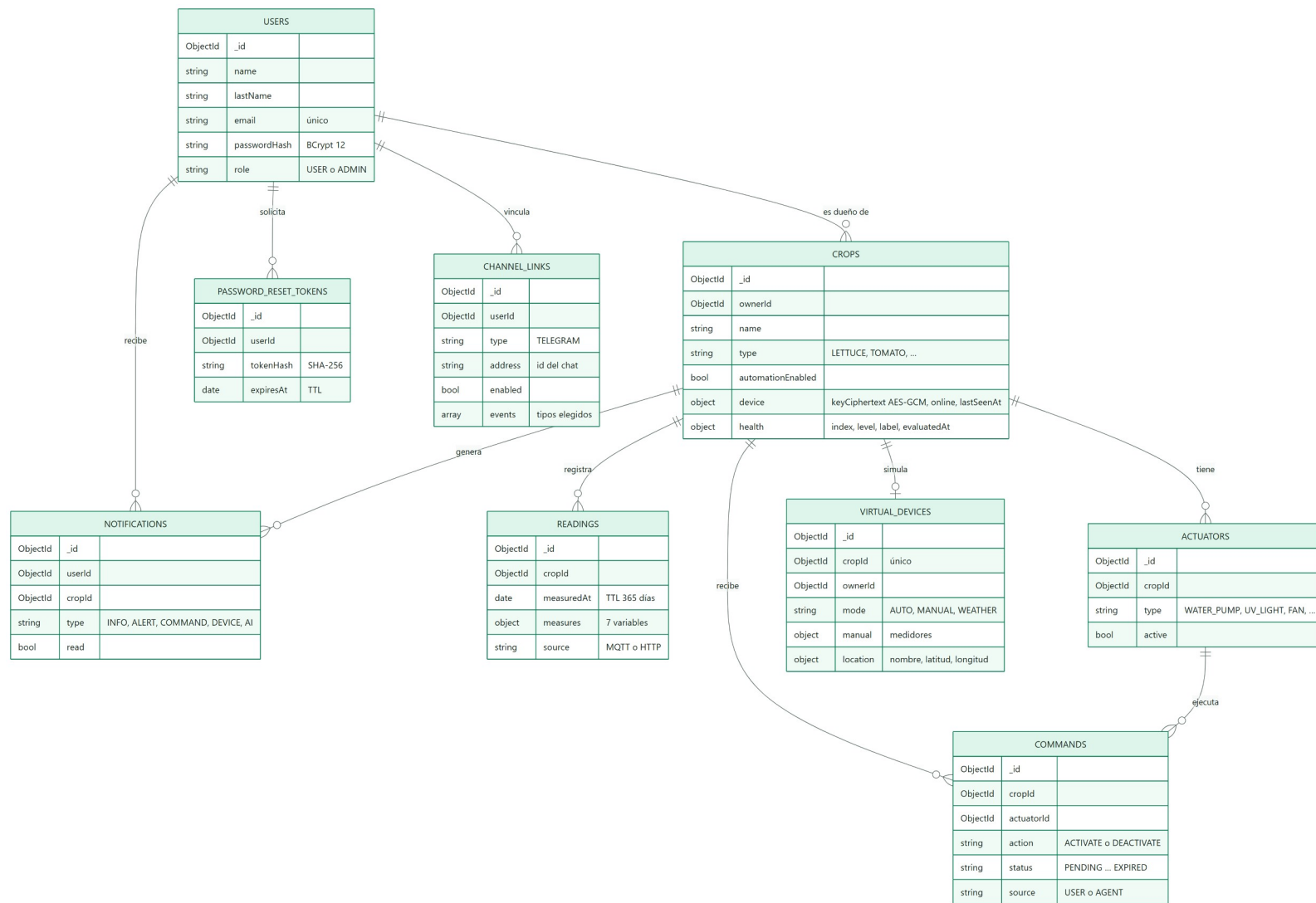


Figura 25 · Colecciones de MongoDB hoy



DISEÑO ORIGINAL	ACTUAL	LECTURA
Session guardada en la base	Sin colección de sesiones	JWT sin estado
Historial con Measures y referencia al cultivo	readings con índice por cultivo y fecha, y TTL de un año	La idea se conservó; se agregaron índices y retención
Cuatro entidades	Colecciones de usuarios, cultivos, lecturas, comandos, notificaciones, vínculos de canales y más	El modelo creció con el dominio, no por anticipado

21.7 Objetos y artefactos

El diseño incluía otros dos diagramas que no tienen un equivalente uno a uno:

DIAGRAMA ORIGINAL	QUÉ MOSTRABA	HOY
Objetos	Instancias de DeviceController, Cultivo, Session, Notificacion y sensores con valores de ejemplo	Las pruebas de la API y de la IA construyen esos mismos objetos con datos válidos en cada ejecución
Artefactos	Navegador, servidor de aplicación, servidor de API, servidor de base de datos con MongoDB y el ESP32, unidos por HTTP	El despliegue continuo y las redes de producción (figuras de despliegue y redes)

El diagrama de objetos quedó a medio terminar: conserva los compartimentos de ejemplo de la herramienta y fechas imposibles como «34/34/2324». El de artefactos, en cambio, ya elegía MongoDB, aunque el acta posterior pedía H2 o MySQL con JPA: la decisión de datos se tomó en el diseño y el acta no la recogió.

LECTURA CRÍTICA

El diseño original acertó en lo que depende del problema (sensores, actuadores, cultivo, historial y un broker) y falló en lo que depende de la tecnología (dónde se decide, cómo se guarda una sesión, cómo se confirma un comando). Es la señal de un buen análisis del dominio con poca experiencia en sistemas distribuidos, y justifica haber rediseñado desde el contrato MQTT hacia afuera.

SECCIÓN 22

Interfaz

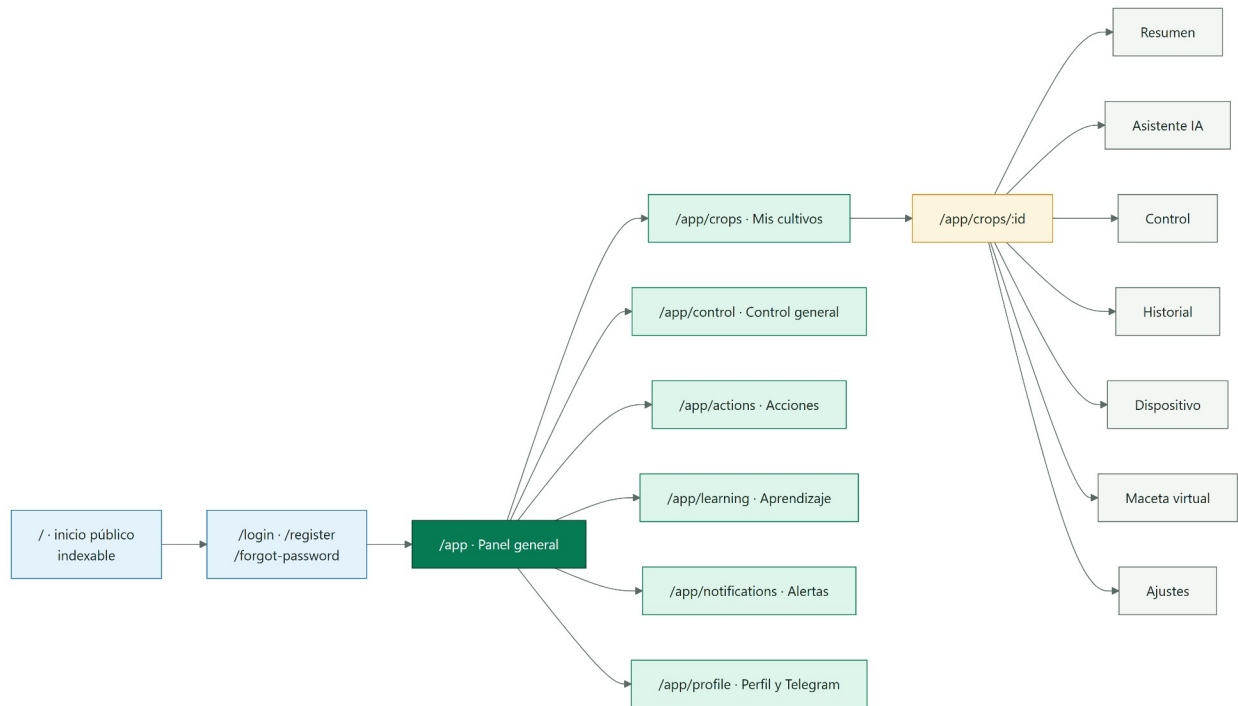


Figura 26 · Mapa de la PWA

PRINCIPIO	CÓMO SE APLICA
Primero el teléfono	Navegación inferior en el teléfono y lateral en el escritorio; la página de aprendizaje solo en escritorio
Lenguaje simple	Textos en español, sin jerga, con explicaciones en cada tarjeta
Accesibilidad	Colores de texto de alto contraste, tablas con encabezados y títulos accesibles, y movimiento reducido respetado en la escena del clima
Identidad	Paleta SmartPot (verdes hoja, azul agua, sol y arcilla) validada para daltonismo en las gráficas
Descubrimiento	Página de inicio indexable con metadatos para buscadores y redes

SECCIÓN 23

Seguridad por diseño

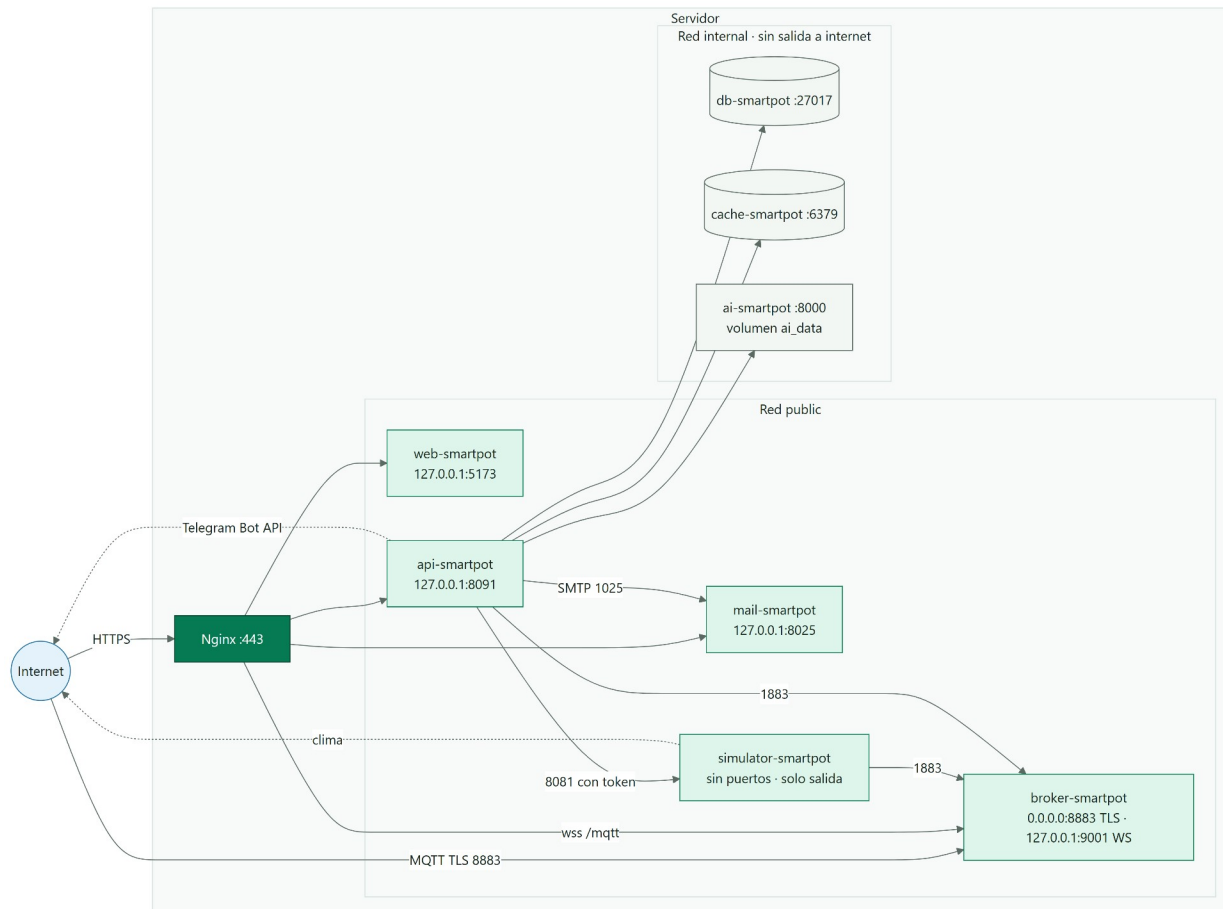


Figura 27 · Redes y puertos en producción

AMENAZA	CONTROL
Suplantación de una maceta	Usuario MQTT por cultivo con clave generada por la API, cifrada con AES-GCM y ACL por tópico
Acceso a cultivos ajenos	Control de dueño en cada servicio, probado con la cadena de seguridad real
Fuerza bruta	BCrypt de costo 12 y límite de peticiones por IP
Escucha en la red	TLS en la web y en la API; MQTT en el puerto 8883 con una autoridad certificadora propia
Exposición de servicios internos	Redes internas de Docker y solo Nginx publicado
Webhook falsificado	Secreto en la cabecera de Telegram
Dependencias vulnerables	Dependabot, CodeQL y SBOM en cada imagen





PARTE V

Construcción

SECCIÓN 24

Organización del trabajo

PRÁCTICA	DETALLE
Repositorios	Uno por componente, más <code>.github</code> para entornos, despliegue, QA y documentación
Idiomas	Código en inglés; interfaz, mensajes, registros y documentación en español
Herramientas	Python con uv, web con pnpm, Java con Maven Wrapper
Commits	En inglés, en imperativo, con un solo propósito y uno o dos archivos
Calidad continua	CI en cada repositorio, CodeQL, Dependabot y QA central
Entornos	Demo de un solo comando, desarrollo y producción con el mismo <code>compose</code> base

SECCIÓN 25

Infraestructura y despliegue

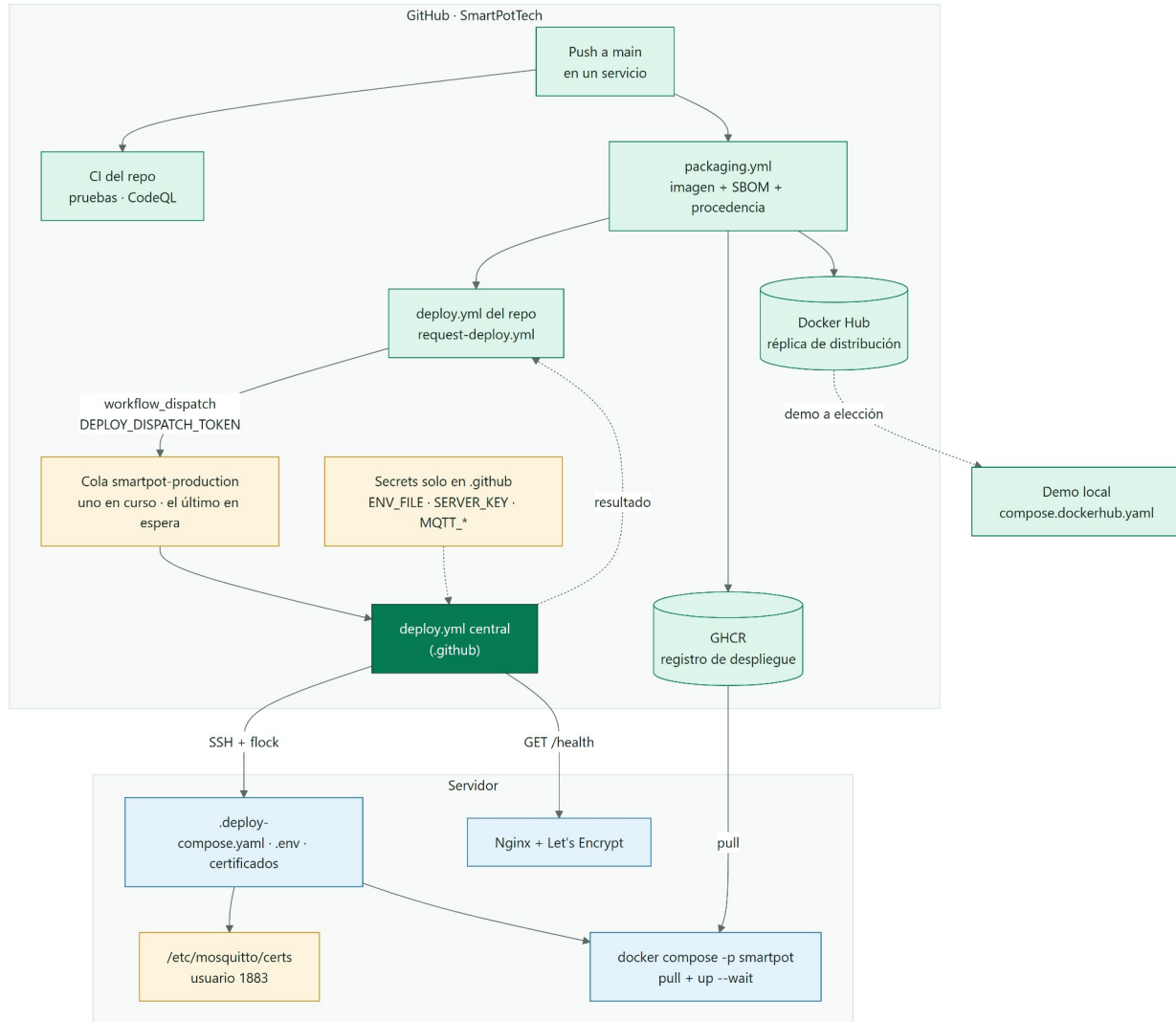


Figura 28 · Despliegue continuo

Cada servicio publica su imagen en GHCR (y en Docker Hub como réplica) y pide el despliegue al workflow central, que corre de a uno: si llegan varios pedidos a la vez, solo el más reciente espera. El servidor descarga las imágenes y recrea solo lo que cambió.



SECCIÓN 26

Asistente de IA

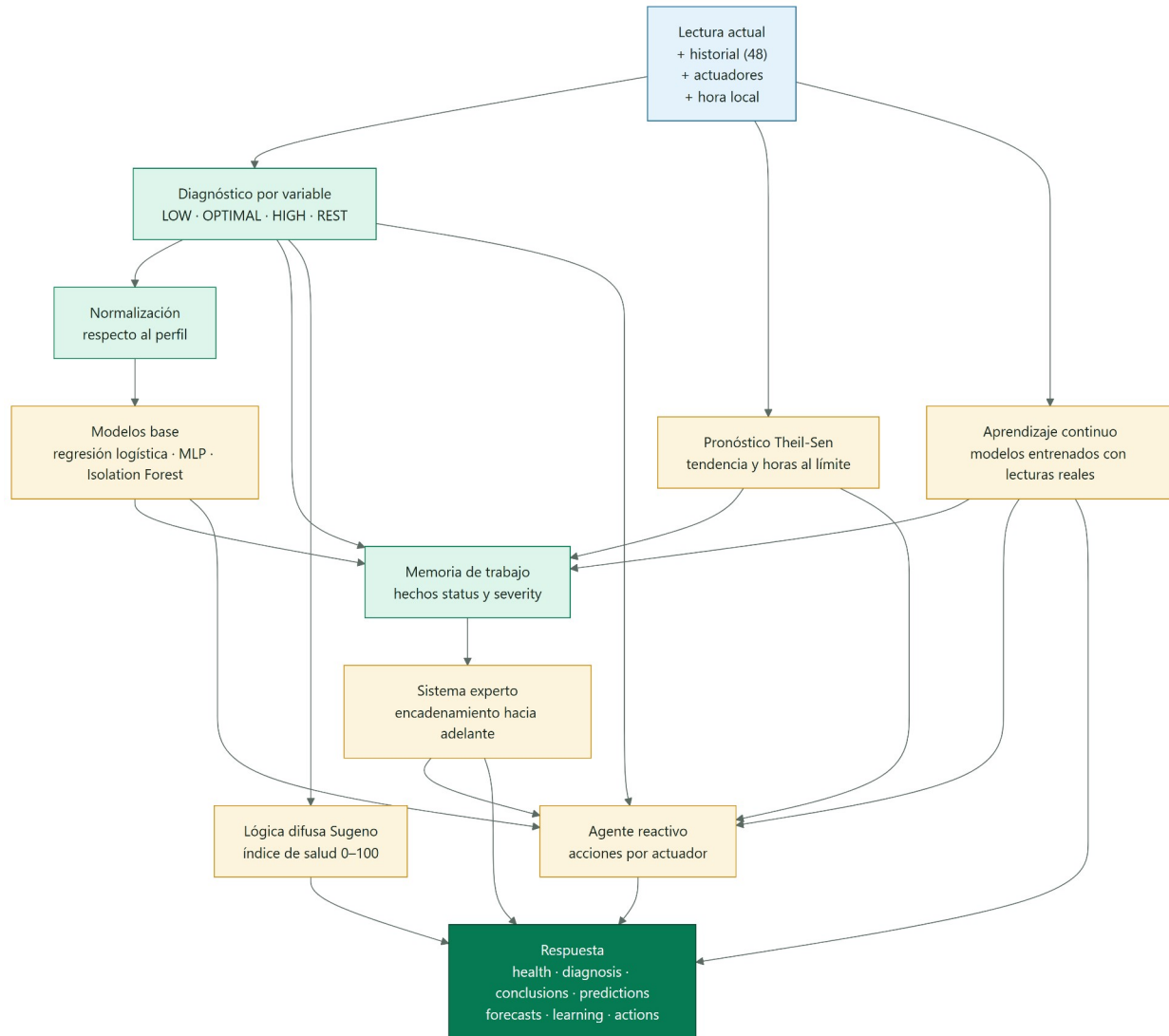


Figura 29 · Técnicas del asistente de IA

La IA se construyó por capas, de lo explicable a lo aprendido: sistema experto con reglas encadenadas, lógica difusa para el índice de salud, pronóstico Theil-Sen de horas hasta el límite, análisis de flota con K-Means y, encima, modelos entrenados con las lecturas reales.

SECCIÓN 27

Integraciones

27.1 Telegram

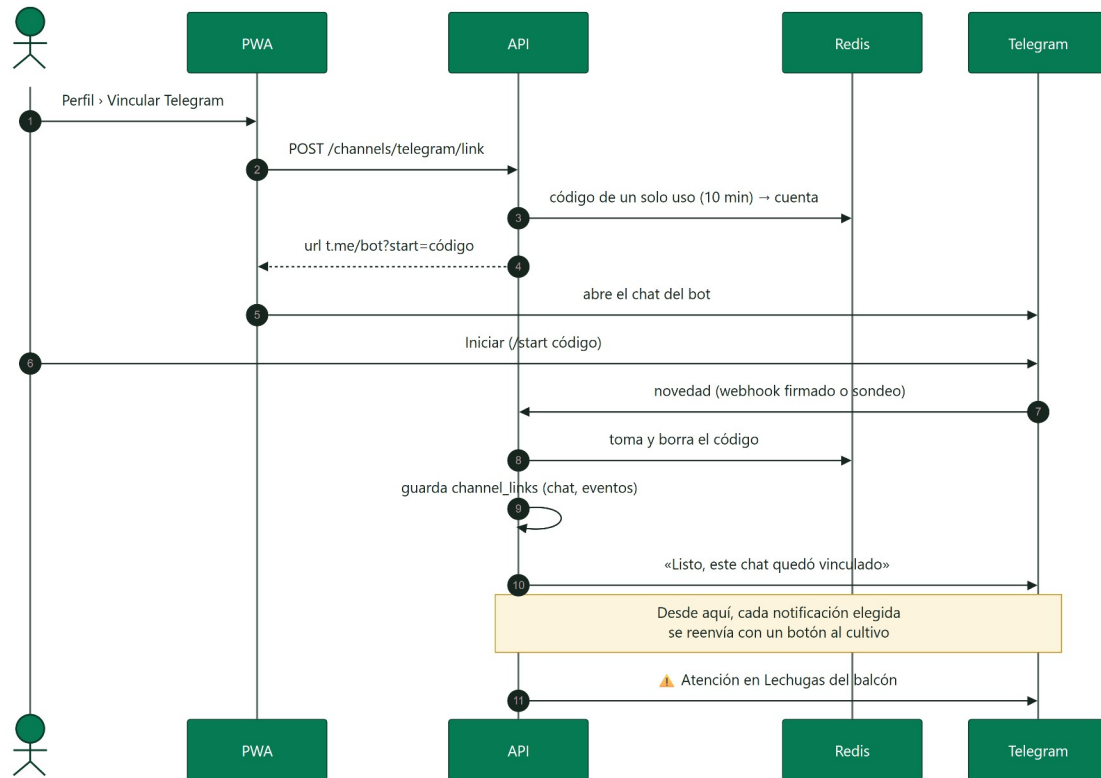


Figura 30 · Vinculación de Telegram

27.2 Macetas virtuales

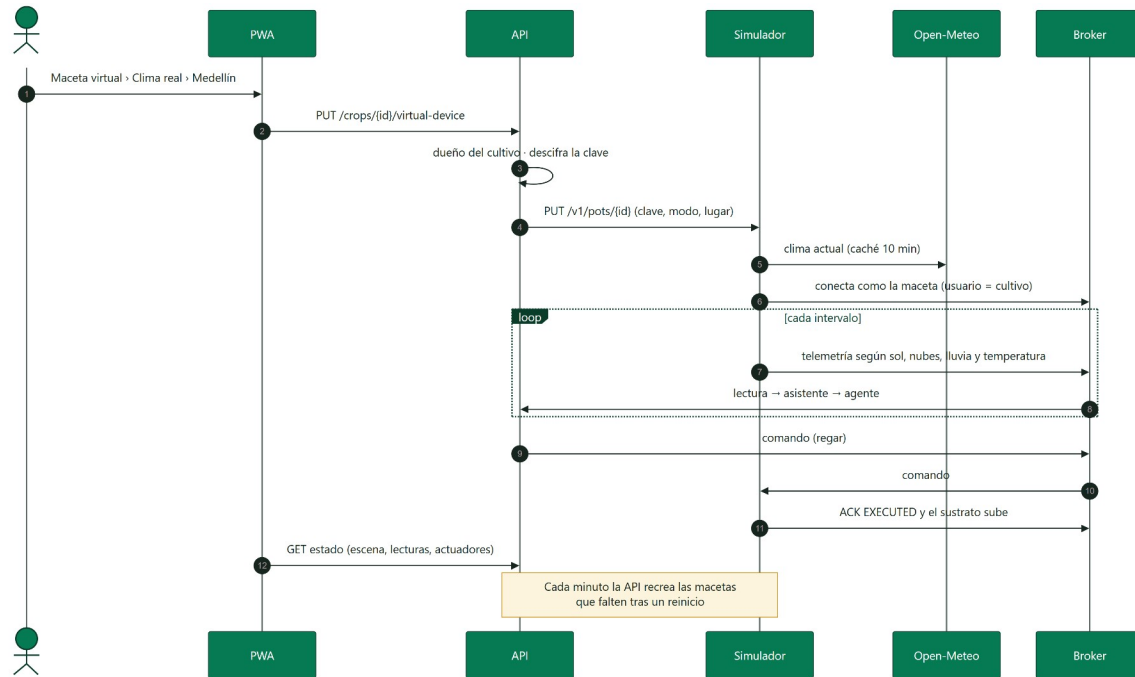


Figura 31 · Maceta virtual con clima real



SECCIÓN 28

Aprendizaje continuo

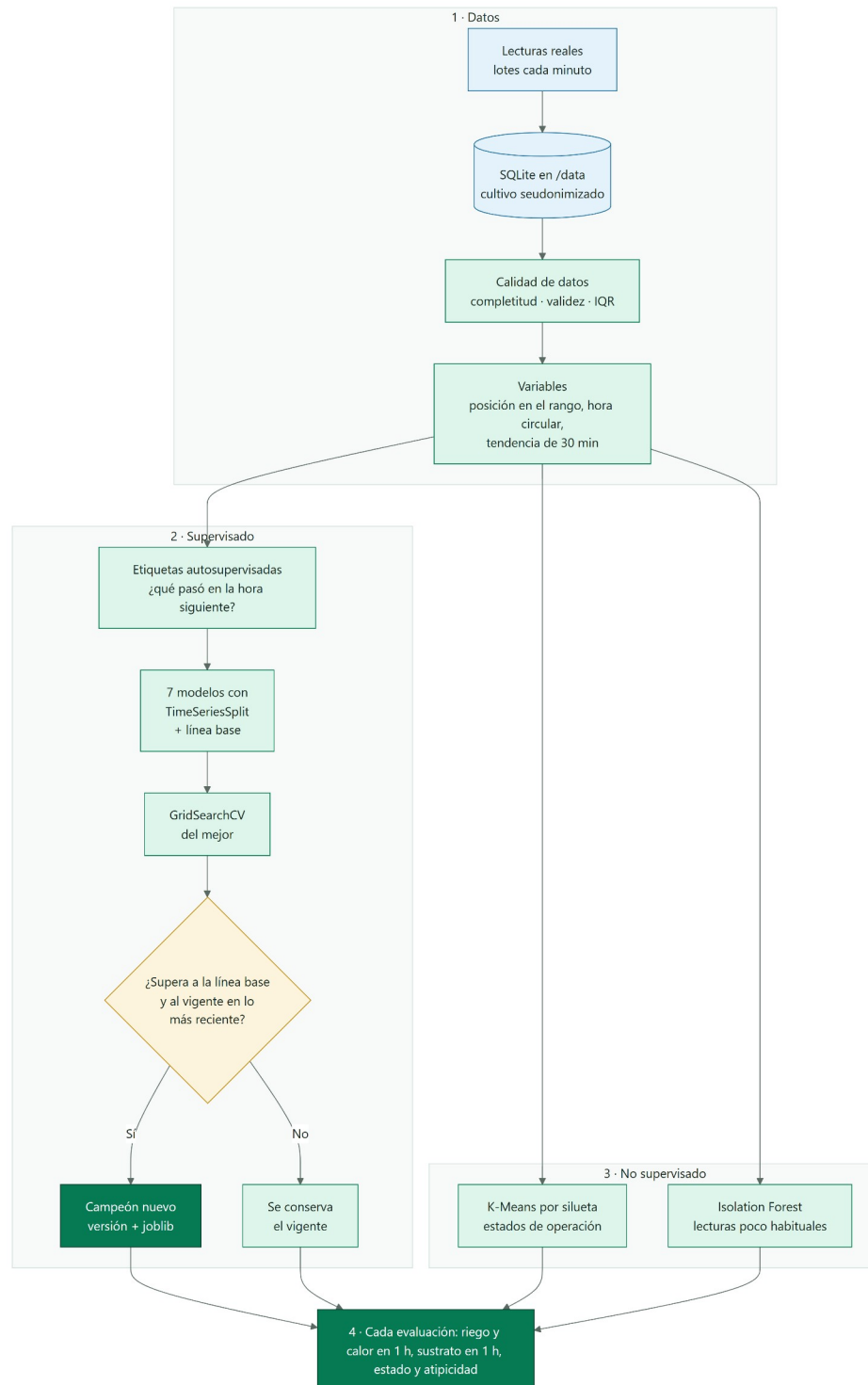


Figura 32 · Aprendizaje continuo con lecturas reales



Cada lectura real llega a la IA con el cultivo seudonimizado. La IA mide su calidad, etiqueta lo que pasó en la hora siguiente, compara siete modelos con validación cruzada temporal frente a una línea base, afina el mejor y solo lo pone en servicio si supera al vigente. Con aprendizaje no supervisado descubre los estados típicos de cada especie y reconoce lo poco habitual.

LECTURA CRÍTICA DE LA CONSTRUCCIÓN

Lo más difícil no fue programar funciones sino sostener la coherencia entre diez repositorios: un contrato que cambia en la API tiene que cambiar en el firmware, el simulador, la IA, la PWA y la documentación. Tres prácticas lo hicieron posible: contratos versionados, un QA que arma la plataforma completa en cada cambio y convenciones iguales en todos los repositorios.



PARTE VI

Pruebas

El acta pedía pruebas unitarias, de integración y de aceptación, con una cobertura mínima y un tiempo de respuesta máximo. La plataforma llegó a una pirámide completa que se ejecuta sola en cada cambio; esta parte muestra cómo está armada, qué criterios del acta cumple y cuáles siguen sin medirse.



SECCIÓN 29

Estrategia

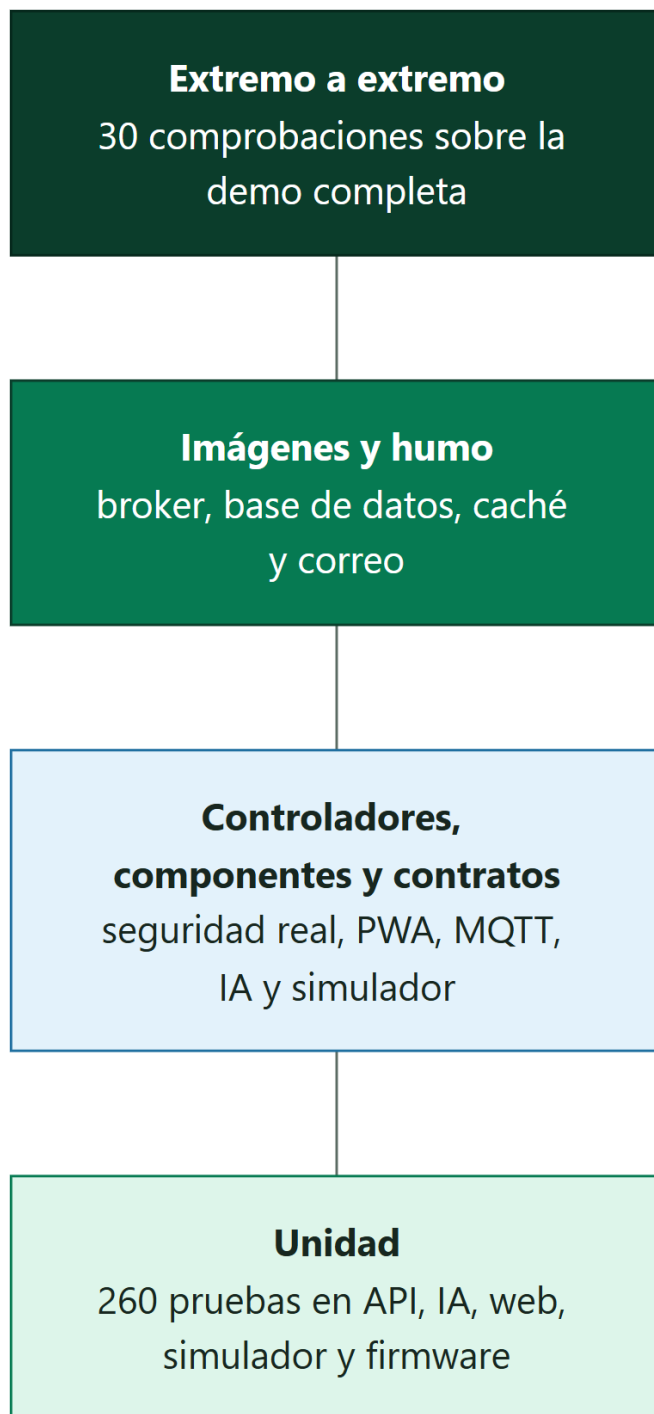


Figura 33 · Pirámide de pruebas

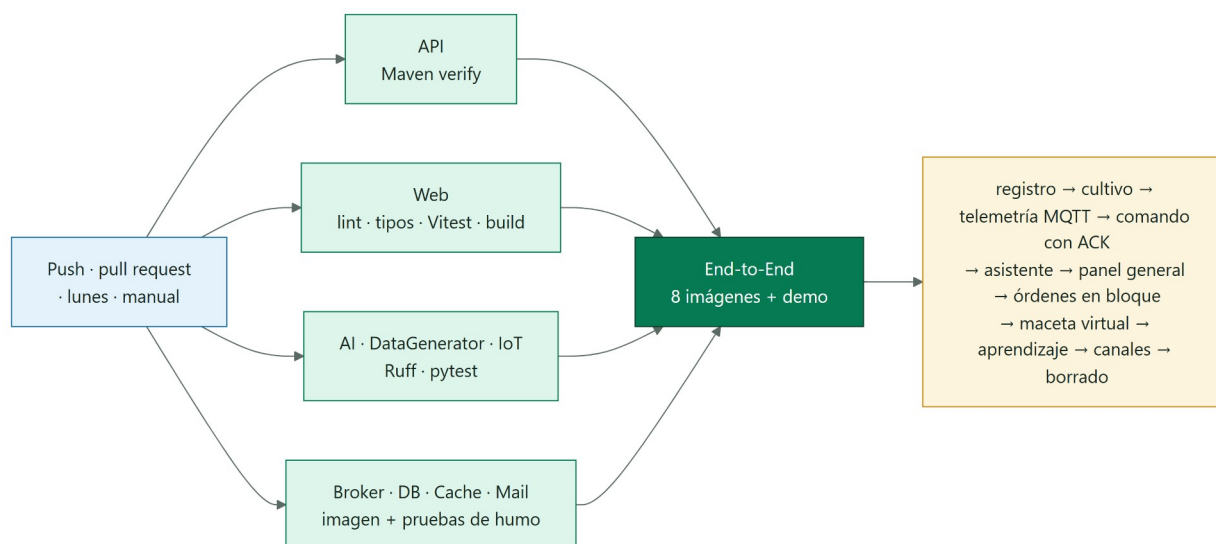


Figura 34 · Workflow de QA

La base son 260 pruebas unitarias y de componentes (106 en la API, 73 en la IA, 45 en la PWA, 23 en el simulador y 13 en el firmware); encima, las pruebas de humo de cada imagen; y arriba, 30 comprobaciones de extremo a extremo sobre la demo completa. El QA corre en cada cambio de la organización, cada lunes y a mano.

SECCIÓN 30

Criterios de aprobación del acta

CRITERIO	ESTADO	CÓMO CERRARLO
Simulación IoT, API con JWT, portal en tiempo real y automatización	Cumplido	Probado por el E2E
Cobertura de al menos 80 % en módulos críticos	Sin medir	Publicar la cobertura en el CI de API, IA y PWA y fijar el umbral por módulo
Latencia de la API menor a 500 ms	Sin medir formalmente	Prueba de carga con varias macetas simuladas y registro del percentil 95
TLS y JWT	Cumplido	Verificado en producción y en las pruebas del broker
Pruebas unitarias, de integración y de aceptación sin defectos críticos	Cumplido	La aceptación por caso de uso está en el recorrido del proyecto
Documentación: requisitos, manual, C4, modelo de datos y despliegue	Cumplido	Estos tres documentos, los READMEs y la guía de la demo

**LECTURA CRÍTICA**

Las pruebas encontraron lo que se buscaba y dejaron pasar lo que no: la caída de producción por las rutas públicas no la detectó ninguna prueba porque ninguna recorría la configuración real. El E2E actual, que se registra e ingresa sobre las imágenes publicadas, cerraría ese hueco. Los dos criterios numéricos del acta (cobertura y latencia) siguen sin medición y son la deuda de pruebas más clara.

SECCIÓN 31

Calidad de los modelos

Las pruebas de la IA entrenan con dos macetas simuladas durante tres días y exigen que cada modelo supere a su línea base en el 20 % de lecturas más recientes: F1 de 0,89 frente a 0,46 para anticipar el secado, F1 de 0,92 frente a 0,49 para anticipar el calor y un error de 1,2 % frente a 4,5 % al estimar la humedad del sustrato en una hora. Son series simuladas: en producción cada especie muestra sus puntajes reales en la página Aprendizaje.



PARTE VII

Mejora continua

SmartPot no terminó con una entrega: siguió mejorando en ciclos cortos que parten de un diagnóstico sobre la plataforma real. Esta parte describe ese ciclo, lo que cada vuelta cambió, los mecanismos que mejoran el sistema sin intervención, los indicadores con los que se medirá el próximo ciclo y las lecciones que deja el proyecto.

SECCIÓN 32

El ciclo de mejora

Cada ciclo empieza con un diagnóstico sobre la plataforma real, no sobre el plan: qué falla, qué está expuesto, qué pide la persona. Termina cuando el cambio está desplegado, probado de extremo a extremo y documentado.

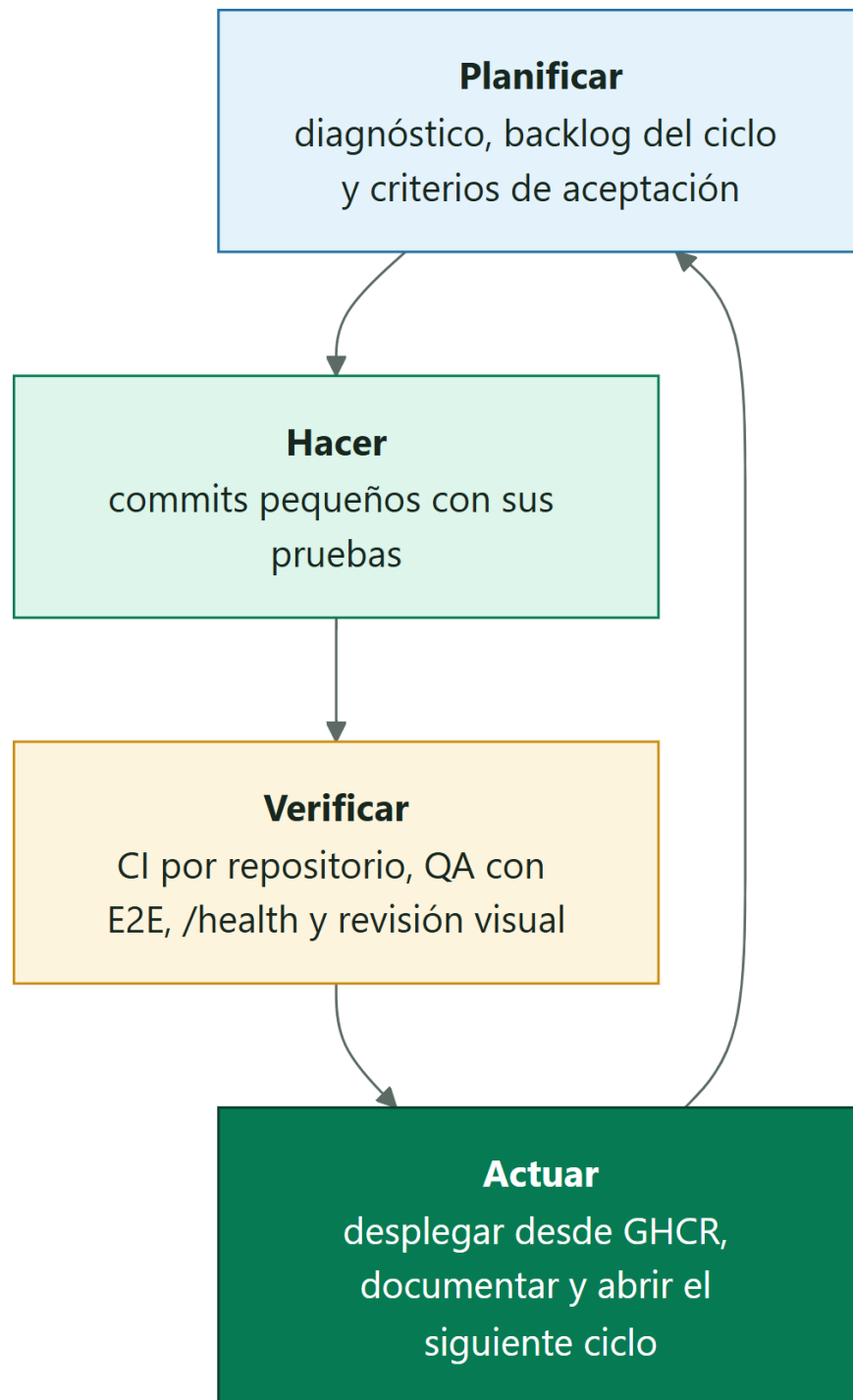


Figura 35 · Ciclo PDCA de SmartPot



SECCIÓN 33

Ciclos de mejora

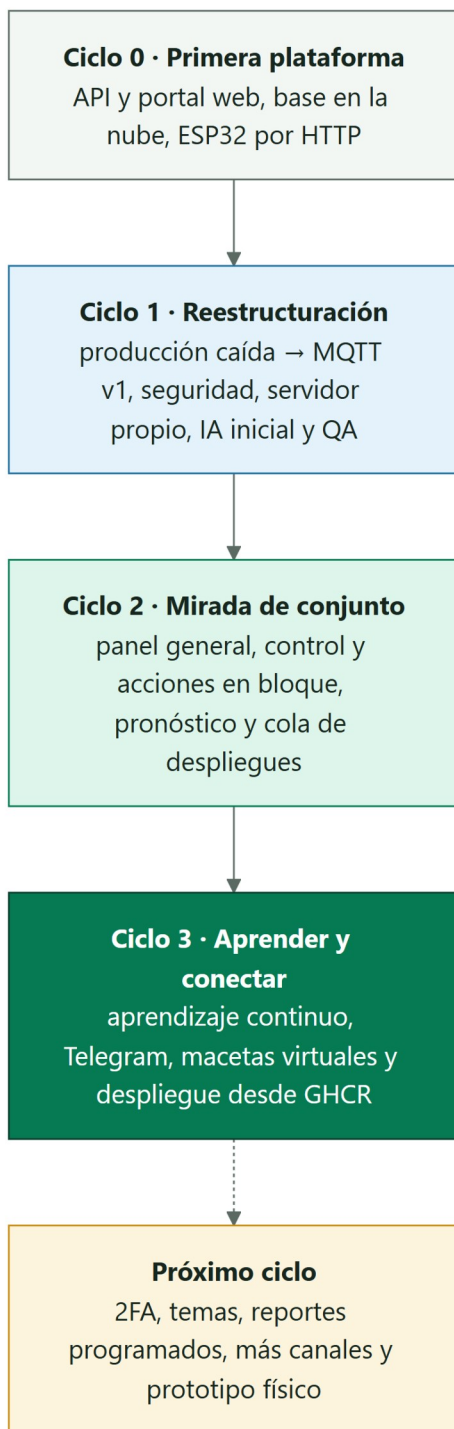


Figura 36 · Ciclos de mejora



CICLO	DISPARADOR	CAMBIOS PRINCIPALES	EVIDENCIA
0 · Primera plataforma	Proyecto de diseño	API, portal web, base en la nube y ESP32	Repositorios desde 2024
1 · Reestructuración	Producción caída, servicios expuestos y rutas sin control de dueño	MQTT v1, servidor propio, seguridad, IA inicial, documentación y QA	Plataforma en smartpot.app; QA con E2E en verde
2 · Mirada de conjunto	Una persona con varios cultivos no tenía vista general	Panel general, control y acciones en bloque, pronóstico y cola de despliegues	Nuevos endpoints y 21 comprobaciones E2E
3 · Aprender y conectar	La IA no mejoraba con el uso y no había avisos fuera de la PWA	Aprendizaje continuo, Telegram, macetas virtuales con clima real y despliegue desde GHCR	260 pruebas y 30 comprobaciones E2E
Próximo	Pendientes de requisitos y de la investigación	Sección 36	—

SECCIÓN 34

Mejora que ocurre sola

MECANISMO	QUÉ MEJORA	FRECUENCIA
Aprendizaje continuo	Los modelos se reentrenan con lecturas reales y solo reemplazan al vigente si lo superan	Cada cierto número de lecturas nuevas por especie
Dependabot	Dependencias al día en todos los repositorios	Semanal en casi todos los ecosistemas; cada actualización pasa por el CI y se revisa antes de fusionar
CodeQL	Detección de vulnerabilidades en el código	En cada cambio
QA programado	La plataforma completa se arma y se prueba aunque nadie haya cambiado nada	Cada lunes

SECCIÓN 35

Indicadores

INDICADOR	DEFINICIÓN	FUENTE	META PROPUESTA
Tiempo en rango	Porcentaje de lecturas dentro del rango ideal de la especie	Lecturas en MongoDB	Subir semana a semana en cultivos con modo automático
Intervenciones manuales	Comandos de la persona frente a los del agente	Colección de comandos	Bajar a medida que el agente aprende



INDICADOR	DEFINICIÓN	FUENTE	META PROPUESTA
Ventaja de los modelos	Puntaje de cada modelo frente a su línea base	Página Aprendizaje	Siempre por encima de la línea base
Latencia de la API	Percentil 95 del tiempo de respuesta	Prueba de carga	Menos de 500 ms (criterio del acta)
Cobertura de pruebas	Porcentaje de líneas cubiertas en módulos críticos	CI de cada repositorio	80 % (criterio del acta)
Frecuencia de despliegue	Despliegues a producción por semana	Workflow central	Al menos uno por ciclo
Cambios fallidos	Despliegues que requieren revertir	Workflow central y /health	Cero
Adopción	Cuentas y macetas activas, en forma agregada y anónima	API	Línea base en el piloto académico

SECCIÓN 36

Backlog del próximo ciclo

PRIORIDAD	TRABAJO	ORIGEN
Must	Medir cobertura y latencia en el CI	Criterios de aprobación del acta
Must	Tiempo en rango e intervenciones como indicadores visibles	Marco lógico
Must	Métricas de adopción agregadas y anónimas	Estudio de mercado
Should	Autenticación en dos pasos	RNF-001
Should	Umbrales propios por cultivo y calibración remota	RF-008 y RF-002
Should	Reportes periódicos programados	RF-012
Could	Actualización en vivo hacia la PWA	RNF-003
Could	Temas y ajustes de visualización	RNF-006
Could	Más canales de notificación	Arquitectura de canales
Later	Prototipo físico y dos ciclos de cultivo comparados	Propuesta de investigación
Later	Redes recurrentes cuando haya meses de datos reales	Propuesta de investigación



SECCIÓN 37

Lecciones aprendidas

DISCIPLINA	LECCIÓN
Formulación	Formular el proyecto como flujo de datos, y no como aparato, fue lo que permitió que el software valiera sin hardware
Evaluación	Un estudio de mercado sin fuente primaria da un buen mapa y malos números; la variable que decide es la conversión y hay que medirla
Gestión	Un plan que no registra avance no detecta desvíos; la evidencia automática (pruebas, QA y despliegues) fue mejor termómetro que el porcentaje declarado
Gestión	Nivelar recursos y aceptar una fecha más tarde es una decisión de gestión, no un fracaso
Análisis	Un requisito que no se puede probar termina redefinido por quien lo construye; conviene escribirlo verificable desde el principio
Diseño	Los contratos primero (MQTT v1 y OpenAPI) desacoplan equipos y repositorios mejor que cualquier red de precedencias
Construcción	Diez repositorios solo son manejables con convenciones idénticas y un QA que los arme juntos
Pruebas	Las pruebas deben recorrer la configuración real, no solo el código
Mejora continua	Ciclos cortos con diagnóstico sobre la plataforma real avanzaron más en semanas que la planeación de un semestre